

Projektdokumentation zur Sommerprüfung 2024

listify.pro

Backend-Infrastruktur für eine iOS / Android App
zur Erstellung und Durchführung von Umfragen.

Prüfungsteilnehmer

Alessandro Ricardo Salomon
Werderstraße 22
68753 Waghäusel

Praktikumsbetrieb

TrendTec UG
Mannheimer Straße 105a
68535 Edingen-Neckarhausen

Projektverantwortlicher

Oliver Kölsch

Ausführungszeitraum

08.05. bis 26.05.2024

Ausbildungsberuf

Fachinformatiker / Anwendungsentwicklung

Inhaltsverzeichnis

Inhaltsverzeichnis.....	A
Abbildungsverzeichnis.....	C
Tabellenverzeichnis.....	D
Verzeichnis der Listings	E
Abkürzungsverzeichnis	F
1 Einleitung	- 1 -
1.1 Projektumfeld.....	- 1 -
1.2 Projektziel.....	- 1 -
1.3 Projektbegründung	- 2 -
1.4 Projektschnittstellen.....	- 2 -
1.5 Projektabgrenzung	- 2 -
2 Projektplanung.....	- 3 -
2.1 Projektphasen und Meilensteine	- 3 -
2.2 Abweichungen vom Projektantrag	- 3 -
2.3 Ressourcenplanung	- 4 -
2.4 Entwicklungsprozess	- 4 -
3 Analysephase	- 4 -
3.1 Ist-Analyse.....	- 4 -
3.2 Wirtschaftlichkeitsanalyse	- 5 -
3.2.1 MVP.....	- 5 -
3.2.2 Projektkosten	- 5 -
3.2.3 Amortisationsdauer	- 6 -
3.3 Anwendungsfälle.....	- 7 -
3.4 Lastenheft	- 7 -
4 Entwurfsphase	- 7 -
4.1. Zielplattform.....	- 7 -
4.2 Architekturdesign.....	- 7 -
4.3 Entwurf der Benutzeroberfläche	- 8 -
4.4 Datenmodell	- 9 -
4.5 Geschäftslogik	- 9 -
4.6 Festlegen der API-Endpunkte	- 9 -
4.7 Maßnahmen zur Qualitätssicherung.....	- 10 -
4.8 Pflichtenheft	- 10 -
5 Implementierungsphase	- 10 -
5.1 Implementierung der Datenstrukturen	- 10 -
5.2 Implementierung der Geschäftslogik	- 12 -
6 Abnahmephase	- 12 -

7 Dokumentation.....	- 13 -
8 Fazit	- 13 -
8.1 Soll-/Ist-Vergleich	- 13 -
8.2 Lessons Learned	- 14 -
8.3 Ausblick	- 14 -
Anhang	i
Anlage 1 Detaillierte Zeitplanung	i
Anlage 2 Zeitplanung in Odoo	i
Anlage 3 Verwendete Ressourcen.....	ii
Anlage 4 Lastenheft (Auszug)	iii
Anlage 5 Benutzeroberfläche.....	vi
Anlage 6 Entity Relationship Diagramm.....	vi
Anlage 7 Datenbankmodell	vii
Anlage 8 Geschäftsprozessdiagramm.....	viii
Anlage 9 Versionskontrolle.....	ix
Anlage 10 Pflichtenheft (Auszug)	ix
Anlage 11 Installierte Symfony Bundles.....	xii
Anlage 12 Fixtures Funktionen (Auszug)	xiii
Anlage 13 Befehl zum Laden der Fixtures in die Datenbank.....	xiii
Anlage 14 Testklasse User Registration	xiv
Anlage 15 Konfiguration JWT.....	xv
Anlage 16 Controller zur Registrierung	xvi
Anlage 17 Anwenderdokumentation.....	xvii
Anlage 18 API-Dokumentation	xx
Anlage 19 Soll-Ist-Vergleich.....	xxi

Abbildungsverzeichnis

Abbildung 1 Auszug Projektstruktur der Entitäten und Repositorys	- 11 -
Abbildung 2 JSON Request	- 12 -
Abbildung 3 Testergebnis der Entität Fragetyp.....	- 13 -
Abbildung 4 Zeitplanung in Odoo	i
Abbildung 5 Wireframe.....	vi
Abbildung 6 Entity Relationship Diagramm.....	vi
Abbildung 7 Datenbankmodell.....	vii
Abbildung 8 Geschäftsprozessdiagramm	viii
Abbildung 9 Versionskontrolle mit Sourcetree	ix
Abbildung 10 Installierte Symfony Bundles	xii
Abbildung 11 Auszug 1 API-Dokumentation	xx
Abbildung 12 Auszug 2 API-Dokumentation	xxi

Tabellenverzeichnis

Tabelle 1 Grobe Zeitplanung	- 3 -
Tabelle 2 Kalkulation Projektkosten	- 6 -
Tabelle 3 Detaillierte Zeitplanung	i
Tabelle 4 Soll-Ist-Vergleich	xxi

Verzeichnis der Listings

Listing 1 Datenbank URL in .env.test.local	- 11 -
Listing 2 Fixtures Funktionen (Auszug)	xiii
Listing 3 Befehl zum Laden der Fixtures in die Datenbank	xiii
Listing 4 Testklasse User Registration	xiv
Listing 5 Konfiguration JWT	xv
Listing 6 Controller zur Registrierung	xvi

Abkürzungsverzeichnis

API: Application Programming Interface

CLI: Command Line Interface

ERD: Entity-Relationship-Diagramm

IaaS: Infrastructure as a Service

JSON: JavaScript Object Notation

JWT: JSON Web Token

MVP: Minimum Viable Product

ORM: Object-relational Mapping

REST: Representational State Transfer

TDD: Test-driven Development

TTL: Time to Live

1 Einleitung

In der vorliegenden Dokumentation wird der Verlauf des IHK-Abschlussprojekts beschrieben, welches vom Autor während seiner Ausbildung/Umschulung zum Fachinformatiker in der Fachrichtung Anwendungsentwicklung bei TrendTec UG durchgeführt wurde

1.1 Projektumfeld

Die TrendTec UG, ein etablierter Anbieter von Softwarelösungen, zielt darauf ab, spezifische technologische Anforderungen verschiedener Branchen zu erfüllen. Das Unternehmen, mit Sitz in Edingen spezialisiert sich auf die Entwicklung maßgeschneiderter Webanwendungen und setzt dabei auf eine fundierte Erfahrung in diesem Bereich. Trotz einer kleinen, spezialisierten Belegschaft von zwei Mitarbeitern steht TrendTec UG für qualitativ hochwertige und innovative Lösungen.

Im Rahmen dieses Projekts agiert TrendTec UG als Auftraggeber für die Entwicklung von „listify.pro“, einer innovativen App, die auf private Anwender abzielt.

1.2 Projektziel

Das Projekt dreht sich um die Entwicklung der "listify.pro" App, eine interaktive Plattform für private Nutzer, die es ermöglicht, Umfragen zu erstellen, zu veröffentlichen und Kommentare von anderen Nutzern zu sammeln. Ziel der App ist es, einen lebendigen und benutzerfreundlichen Raum für den Meinungsaustausch zu schaffen.

Das Projekt beinhaltet neben der grundlegenden Umfragefunktionalität mit verschiedenen Frage-Typen (Offen, Mehrfachauswahl, Einzelauswahl) auch die Entwicklung einer erweiterten Statistikfunktion. Diese ermöglicht es den Nutzern, detaillierte Analysen und Auswertungen der Umfrageergebnisse durchzuführen. Dadurch gewinnen sie tiefere und fundiertere Einsichten in die gesammelten Meinungen und Feedbacks.

Im Rahmen der Umsetzung wurden folgende Technologien in Betracht gezogen:

Datenbank: Eine MySQL-Datenbank speichert sicher und skalierbar Benutzerdaten, Umfragedaten und Kommentare.

API: Die REST API (Representational State Transfer- Application Programming Interface) wird mit Symfony 7.0 ¹entwickelt und verwaltet die Interaktionen zwischen App und Datenbank. Sie stellt Endpunkte für Umfragen, Authentifizierung und statistische Auswertungen bereit.

Frontend: Das Frontend wird mit React Native von einem separaten Team entwickelt, um eine plattformübergreifende Lösung für iOS und Android zu gewährleisten. Die enge Zusammenarbeit mit dem Backend-Team sichert eine nahtlose Integration.

Backend: Das Backend, ebenfalls in Symfony 7.0 entwickelt, übernimmt die Geschäftslogik und Datenverarbeitung. Sicherheitsmaßnahmen wie Token-basierte Authentifizierung sind integriert.

Zusammenfassend lässt sich sagen, dass das Ziel dieses Projekts darin besteht, eine Plattform zu schaffen, die es erleichtert, Meinungen zu sammeln und zu analysieren, um besser informierte Entscheidungen zu treffen oder Einblicke in bestimmte Zielgruppen zu erhalten.

¹ <https://symfony.com/>

1.3 Projektbegründung

Die Entwicklung der "listify.pro" App wurde durch die Erkenntnis motiviert, dass aktuell verfügbare Umfrage-Tools wie doodle.com² zwar weit verbreitet, jedoch in ihren Funktionen begrenzt sind und oft durch Werbung finanziert werden, was das Nutzererlebnis beeinträchtigen kann. Während der Recherchephase für das Projekt wurde keine App auf dem Markt gefunden, die ohne Werbefinanzierung betrieben wird und gleichzeitig erweiterte Funktionen wie detaillierte statistische Auswertungen und interaktive Kommentarfunktionen bietet.

Die "listify.pro" App adressiert diese Lücke, indem sie eine Plattform bietet, welche die Nutzerbedürfnisse nach einer reinen Nutzererfahrung ohne störende Werbung erfüllt und zusätzliche, wertvolle Funktionalitäten für eine gründlichere Meinungserhebung und Datenauswertung integriert. Die App erhöht somit die Effizienz in der Datenanalyse und Meinungssammlung, was besonders für private Anwender, die tiefere Einblicke oder umfassendere Rückmeldungen suchen, von großem Vorteil ist.

Weiterhin trägt dieses Projekt zur Zeitersparnis bei den Nutzern bei, da es ihnen ermöglicht, schnell und einfach Umfragen zu erstellen und auszuwerten, ohne dabei durch Werbung unterbrochen zu werden. Diese Aspekte machen das Projekt nicht nur sinnvoll, sondern auch nachhaltig und benutzerfreundlich, was die Motivation hinter der Entwicklung von "listify.pro" weiter unterstreicht.

1.4 Projektschnittstellen

Das Projekt umfasst folgende Schnittstellen:

Der Projektleiter ist für die Koordination aller Projektaktivitäten, die Kommunikation zwischen den Teams sowie die Einhaltung der Projektzeitpläne verantwortlich.

Die Geschäftsführung wird regelmäßig über den Projektfortschritt sowie über Entscheidungen, die das Projekt beeinflussen, informiert.

Das Frontend-Team ist in regelmäßigen Abständen über den Projektfortschritt zu informieren, um eine nahtlose Integration des Backends und des Frontends sowie die korrekte Umsetzung aller Funktionalitäten zu gewährleisten.

Praktikanten sind in den Code-Review-Prozess eingebunden, um sicherzustellen, dass der Code den Qualitätsstandards entspricht und gleichzeitig eine Lernmöglichkeit für die Praktikanten bietet.

1.5 Projektabgrenzung

Im Rahmen der Projektplanung war ich vollständig in die entsprechenden Prozesse des Gesamtprojekts involviert. Die Dokumentation bezieht sich in der Phase der Implementierung auf das Backend.

Im Bereich der Frontend-Entwicklung ist die Entwicklung des Frontends der App nicht Teil des vorliegenden Projekts, welches durch ein separates Team durchgeführt wird. Ich habe an der Planung mitgewirkt.

Hosting-Dienste: Die Auswahl von Hosting-Diensten und anderen technischen Aspekten wird möglicherweise an Drittanbieter ausgelagert und ist nicht Teil dieses Projekts.

Datenbank-Entwicklung: Die Verantwortung für die Datenbank-Entwicklung liegt in meinem Zuständigkeitsbereich. Dies umfasst das Design der Datenbankstruktur sowie die Implementierung der Datenbank.

²<https://doodle.com/de>

API-Entwicklung: Die Entwicklung der API zur Kommunikation zwischen Frontend und Backend ist ebenfalls Teil meines Aufgabenbereichs. Dies umfasst die Spezifikation, Implementierung, Dokumentation und Bereitstellung der API, die benötigt wird, um die Umfragefunktionalitäten und die erweiterten Statistikfunktionen zu unterstützen.

2 Projektplanung

Dieses Kapitel zeigt die genaue Planung des Projektes. Dazu gehört die Auswahl eines geeigneten Vorgehens bei der Entwicklung und die Auswahl aller für die Umsetzung benötigten Ressourcen.

2.1 Projektphasen und Meilensteine

Das Projekt wird in einem definierten Zeitraum und unter spezifischen Rahmenbedingungen durchgeführt. Der geplante Bearbeitungszeitraum für das Projekt ist vom 08.05.2024 bis zum 26.05.2024. Es wird intern von einem Auszubildenden/Umschüler durchgeführt, der eng mit dem Entwicklungsteam zusammenarbeitet. Das Projektteam besteht aus zwei Mitarbeitern der TrendTec UG

Die tägliche Arbeitszeit ist aufgeteilt in Büro- und Home-Office-Zeiten. Konkret bedeutet dies:

- **Bürozeit:** 8:00 bis 13:00 Uhr
- **Home-Office-Zeit:** 14:00 bis 17:00 Uhr

Diese Rahmenbedingungen ermöglichen eine flexible und dennoch strukturierte Arbeitsweise, die es erlaubt, sowohl in direkter Zusammenarbeit mit dem Team als auch in einer ruhigeren Home-Office-Umgebung effektiv zu arbeiten.

Projektphase	Geplante Zeit
Planung und Konzeption	27 h
Programmierung	41 h
Erstellen der Dokumentation	10 h
Abnahme	1 h
Gesamt	80 h

Tabelle 1 Grobe Zeitplanung

2.2 Abweichungen vom Projektantrag

Im Vergleich zum ursprünglichen Projektantrag wurden einige Änderungen in der Zeitplanung (Tabelle 3 Detaillierte Zeitplanung) vorgenommen. Diese Änderungen sind notwendig, um eine detailliertere und realistischere Planung zu gewährleisten und den spezifischen Anforderungen des Wasserfallmodells besser gerecht zu werden. Die Gesamtzeit des Projekts bleibt bei 80 Stunden, jedoch wird die Aufteilung und Strukturierung der Phasen präziser definiert.

Die Erstellung einer detaillierten Anforderungsanalyse stellt einen essenziellen Schritt im Rahmen der Projektplanung dar. Die Analyse ermöglicht die klare Definition der Bedürfnisse und Ziele des Projekts sowie die Identifikation potenzieller Risiken. Die zusätzlichen Stunden für die Ist-Analyse und die Wirtschaftlichkeitsanalyse tragen zu einer fundierten Grundlage für das Projekt bei.

Auch die Erstellung des Pflichtenhefts, des Geschäftsprozessdiagramms sowie die Festlegung der API-Endpunkte stellen wesentliche Schritte im Entwurfsprozess dar, da auf diese Weise klare Anforderungen und Spezifikationen für die Implementierung definiert werden können. Eine präzisere und fokussierte Entwurfsphase ermöglicht folglich eine bessere Grundlage für die anschließende Implementierung.

2.3 Ressourcenplanung

Für die erfolgreiche Umsetzung des Projekts zur Entwicklung einer Backend-Infrastruktur für eine Umfrage-App sind sowohl materielle als auch personelle Ressourcen notwendig. Die Planung umfasst Hard- und Software-Ressourcen sowie das beteiligte Personal.

Bei der Auswahl der verwendeten Software wurde besonderer Wert daraufgelegt, die Kosten zu minimieren. Diesbezüglich wurde bei der Entscheidung für eine bestimmte Software darauf geachtet, dass die Lizenzgebühren niedrig sind und das notwendige Fachwissen bereits vorhanden ist. Dadurch kann das Projekt effizient und kostengünstig umgesetzt werden. Bei Odoo³ fallen Gebühren an, die jedoch nicht sehr ins Gewicht fallen und daher vernachlässigt werden können.

Das Projektteam besteht aus einem Auszubildenden/Umschüler Fachinformatiker für Anwendungsentwicklung, Praktikanten, einem Projektleiter. Die detaillierte Auflistung der benötigten Ressourcen finden Sie in den Anlagen dieser Dokumentation (Anlage 3 Verwendete Ressourcen Seite ii).

2.4 Entwicklungsprozess

Die Entwicklung einer Backend-Infrastruktur für eine Umfrage-App erfolgt gemäß dem Wasserfall-Entwicklungsprozess. Dieser Prozess eignet sich insbesondere für Projekte, bei denen die Aufgabenpakete sowie das Ziel und die Anforderungen von Anfang an klar definiert sind. Da das Projekt im Wesentlichen von einer Person umgesetzt wird, bietet der Wasserfallprozess eine strukturierte und lineare Vorgehensweise. Die nacheinander erfolgende Bearbeitung jeder Phase des Projekts fördert eine klare Trennung der einzelnen Projektphasen und erlaubt eine konzentrierte Bearbeitung jeder Phase.

In Oodo wurden die einzelnen Phasen eingetragen, wodurch ein Überblick über die anstehenden Aufgaben gewährleistet wird (Abbildung 4 Zeitplanung in Oodo).

3 Analysephase

In dieser Phase erfolgt eine Betrachtung des Projekts hinsichtlich des Ist-Zustandes, der Wirtschaftlichkeit sowie der Anwendungsfälle.

Innerhalb des Lastenhefts werden darüber hinaus weitere Spezifikationen erläutert.

3.1 Ist-Analyse

Derzeit steht keine benutzerfreundliche Plattform zur Erstellung und Durchführung von Umfragen zur Verfügung. Dies führt zu einer suboptimalen Benutzererfahrung und erschwert die Erfassung und Analyse von Feedback. Bestehende Programme und Systeme sind entweder zu komplex in der Handhabung oder bieten nicht die notwendigen Funktionen, um den Anforderungen gerecht zu werden.

Die Projektausgangssituation ist dadurch gekennzeichnet, dass noch keine Programmierung vorgenommen wurde und keine bestehende Infrastruktur für die Umfrage-App existiert. Alle erforderlichen Komponenten und Funktionen müssen von Grund auf entwickelt werden, was die Möglichkeit bietet, eine maßgeschneiderte Lösung zu schaffen, die exakt auf die Bedürfnisse und Anforderungen der Nutzer abgestimmt ist.

Das Wettbewerbsumfeld ist ähnlich zu dem von Doodle⁴. Doodle stellt bereits eine Plattform für die Erstellung und Verwaltung von Umfragen und Terminabstimmungen bereit. Um im

³ https://www.odoo.com/de_DE

⁴ <https://doodle.com/de>

Wettbewerb zu bestehen, muss die geplante Umfrage-App durch Benutzerfreundlichkeit, Flexibilität und zusätzliche Funktionen überzeugen.

Die geplante Lösung soll die Lücke schließen und eine moderne, intuitive Umfrage-Plattform bereitstellen. Die Erstellung von Umfragen mit verschiedenen Fragearten und Antwortoptionen, das Teilen von Umfragen über Links, die Echtzeit-Erfassung von Antworten sowie die Integration einer Kommentarfunktion zur direkten Interaktion zwischen den Teilnehmern sind Bestandteile der geplanten Lösung.

3.2 Wirtschaftlichkeitsanalyse

Das Projekt zur Entwicklung einer Backend-Infrastruktur für eine Umfrage-App wird zunächst als kostenloses Angebot eingeführt. Dieses kostenlose Modell wird drei verschiedene Fragetypen umfassen, die den Nutzern zur Verfügung stehen. Ziel ist es, eine möglichst breite Nutzerbasis zu erreichen und erste Erfahrungen mit dem Produkt zu sammeln.

3.2.1 MVP

Das Projekt wird als Minimum Viable Product (MVP) initiiert, um die Akzeptanz des Marktes sowie die tatsächlichen Bedürfnisse der Nutzer zu eruieren. Durch die Bereitstellung einer kostenlosen Version mit eingeschränkten Funktionen kann das Unternehmen wertvolles Feedback sammeln und die App kontinuierlich verbessern. Das Abonnementmodell ermöglicht den Nutzerinnen und Nutzern den Zugriff auf erweiterte Funktionen, die über die Basisversion hinausgehen.

Des Weiteren werden neue Fragetypen zur Erstellung von Umfragen, beispielsweise mit Datum für Termine, angeboten.

Die Möglichkeit zur Erhebung von Nutzerfeedback und zur Verbesserung der App auf Basis tatsächlicher Nutzung stellt einen wichtigen wirtschaftlichen Vorteil dar.

Ein weiteres Argument für das Projekt stellt der Aufbau einer Nutzerbasis dar, die potenziell in zahlende Kunden umgewandelt werden kann.

Nach erfolgreicher Markteinführung und positiver Resonanz ist davon auszugehen, dass das Projekt langfristig rentabel wird. Das Abonnementmodell generiert regelmäßige Einnahmen und stellt gleichzeitig einen Anreiz für die kontinuierliche Verbesserung und Erweiterung der App dar.

3.2.2 Projektkosten

Die Projektkosten beinhalten die Entwicklung, Absprachen, Code Reviews sowie die Abnahme des Projektes und der Dokumentation. Die vorliegende Kalkulation basiert auf folgenden Annahmen:

Die Kosten pro Stunde wurden aufgrund der Qualifikation und der Verantwortung des jeweiligen Mitarbeiters festgelegt. Der Stundensatz für den Umschüler/Auszubildenden beträgt (12€ pro Stunde Umschüler). Der Stundensatz für Auszubildende/Umschüler basiert auf der Rückrechnung des Stundenlohns auf Grundlage des Einkommens während der Umschulung. Dies gilt ebenfalls für Praktikanten. Für erfahrene Mitarbeiter und Projektleiter wird ein Stundensatz von 150 € pro Stunde Mitarbeiter/Projektleiter angesetzt, was die höhere Verantwortung und das erforderliche Fachwissen widerspiegelt. Der Satz setzt sich aus dem Gehalt, den Lohnnebenkosten sowie einem Gewinnzuschlag zusammen.

Die Entwicklungskosten für das Frontend werden auf 2.500 € geschätzt. Diese sind erforderlich, um sowohl die Gesamtkosten als auch die Amortisationsdauer zu berechnen.

Diese detaillierte Aufschlüsselung (Tabelle 2 Kalkulation Projektkosten) ermöglicht eine transparente und nachvollziehbare Darstellung der anfallenden Kosten und hilft, das Budget effizient zu planen und zu überwachen.

Vorgang	Mitarbeiter	Zeit	Kosten pro Stunde	Gesamt
Entwicklungskosten	1 x Auszubildender/Umschüler	80h	12 €	960 €
Absprache Projektleiter	1 x Mitarbeiter	2h	150 €	300 €
Code Review	1 x Praktikant	2h	12 €	24 €
Absprache Frontend	2 x Mitarbeiter	1h	300 €	300 €
Abnahme	1 x Mitarbeiter	2h	150 €	300 €
Abnahme Dokumentation	1 x Mitarbeiter	1h	150 €	150 €
Entwicklung Frontend				2500€
Gesamt				4534 €

Tabelle 2 Kalkulation Projektkosten

3.2.3 Amortisationsdauer

Die Amortisationsdauer gibt an, wie lange es dauert, bis sich die Investition in das Projekt durch die erzielten Einnahmen rentiert. Die Rechnungen basieren auf einem Abo-Modell mit einem Preis von 2,99 € pro Monat. Der Preis ist eine Vorgabe der Geschäftsführung. Es werden drei Szenarien betrachtet. Um einen Anhaltspunkt zu bekommen. Serverkosten werden in dieser Rechnung nicht berücksichtigt, um die Rechnung übersichtlich zu gestalten.

Amortisation in 6 Monaten: Um die Projektkosten innerhalb eines Zeitraums von sechs Monaten zu amortisieren, ist es erforderlich, die Anzahl der erforderlichen Abonnements zu berechnen.

$$\text{Anzahl Abos} = \frac{\text{Projektkosten}}{\text{Abo-Preis pro Monat} \times 6}$$

$$253 = \frac{4534 \text{ €}}{2,99 \text{ €} \times 6}$$

Amortisation bei 200 Abos: Die Fragestellung lautet, wie lange es in Monaten dauert, bis sich das Projekt bei 200 Abonnements amortisiert. Hierbei wird von einem linearen Wachstum der Abos und der Abo-Anzahl von 200 im Mittelwert ausgegangen.

$$\text{Amortisationsdauer} = \frac{\text{Projektkosten}}{\text{Anzahl Abos} \times \text{Abo-Preis pro Monat}}$$

$$7,6 \sim \frac{4534 \text{ €}}{200 \times 2,99 \text{ €}}$$

Amortisation bei 5000 Abos: Die Fragestellung lautet, wie lange es in Tagen dauert, bis sich das Projekt bei 5000 Abonnements amortisiert. Hierbei wird von einem linearen Wachstum der Abos und der Abo-Anzahl von 5000 im Mittelwert ausgegangen.

$$\text{Amortisationsdauer} = \frac{\text{Projektkosten}}{\text{Anzahl Abos} \times \text{Abo-Preis pro Tag}}$$

$$9,1 \sim \frac{4534 \text{ €}}{5000 \times 0,1 \text{ €}}$$

3.3 Anwendungsfälle

Im Lastenheft sind die Anwendungsfälle als User Stories definiert. Diese User Stories beschreiben detailliert die Anforderungen und Erwartungen der Benutzer an die Applikation und dienen als Grundlage für die Entwicklung der Anwendung sowie für die Definition der Endpunkte der API, um diese Dienste bereitzustellen. (A 4.3 Funktionale Anforderungen Seite iv)

Die im Lastenheft dokumentierten User Stories weisen mehrere Vorteile auf:

Die User Stories gewährleisten eine klare und verständliche Beschreibung der Anforderungen aus der Perspektive der Benutzer. Sie stellen eine entscheidende Grundlage für die Planung und Entwicklung der App sowie für die Bereitstellung der erforderlichen API-Funktionalitäten dar.

3.4 Lastenheft

Das Lastenheft definiert die Anforderungen an die Funktionalität der App aus Kundensicht. Eine detaillierte Version des Lastenhefts ist in Anlage 4 Lastenheft Seite iii zu finden.

4 Entwurfsphase

Nach Abschluss der Analysephase wurden Entwürfe für die Architektur und das Datenmodell erstellt. Des Weiteren wurden Maßnahmen zur Qualitätssicherung festgelegt und ein Pflichtenheft erstellt, welches die technische Umsetzung der fachlichen Anforderungen aus dem Lastenheft beinhaltet.

4.1. Zielplattform

Die zu implementierende Lösung umfasst eine REST-API für eine hybride Applikation. Das erforderliche Backend wird mit Symfony 7.0 umgesetzt, während die Datenhaltung mittels einer MySQL-Datenbank erfolgt.

Die Entscheidung für diese Technologien bzw. Framework basiert auf der Annahme, dass im Unternehmen die größte Erfahrung in diesem Bereich vorliegt. Ein weiterer Vorteil besteht in der Beständigkeit der Nutzung von Symfony im Unternehmen. Dadurch ist es möglich, für andere Entwickler schneller einen Überblick über das Projekt zu erhalten.

Die Entscheidung über die genaue Serverplattform ist noch nicht finalisiert. Es besteht die Möglichkeit, zunächst auf den firmeneigenen Server und zu einem späteren Zeitpunkt beispielsweise zu einem IaaS-Anbieter (Infrastructure as a Service) zu wechseln. Dies ermöglicht eine vereinfachte Skalierung bei einem Anstieg der Nutzerzahlen.

4.2 Architekturdesign

Das Architekturdesign des Projekts folgt modernen Prinzipien und Best Practices, um eine flexible, skalierbare und effiziente Anwendung zu gewährleisten. Im Mittelpunkt steht eine REST-API, die das Open-Closed-Prinzip beachtet und die Kommunikation zwischen Client und Server ermöglicht. Zusätzlich werden Websockets für Echtzeitkommunikation verwendet.

Die REST-API dient als zentrale Schnittstelle für die Kommunikation zwischen dem Frontend und Backend der Anwendung. Die REST-API ermöglicht den Zugriff auf verschiedene Ressourcen und Funktionen der Anwendung durch standardisierte HTTP-Methoden (GET, POST, PUT, PATCH, DELETE).

Das Open-Closed-Prinzip besagt, dass neue Endpunkte und Funktionen hinzugefügt werden können, ohne den bestehenden Code zu ändern. Bestehende Endpunkte und Funktionalitäten werden nicht verändert, um sicherzustellen, dass der bereits getestete und verifizierte Code stabil bleibt.

Das Frontend der Anwendung, welches in einem Webbrowser und in einer mobilen Applikation läuft, sendet Anfragen an die API und zeigt die erhaltenen Daten an.

Das Backend verarbeitet die Anfragen, führt die Geschäftslogik aus und liefert die entsprechenden Daten oder Aktionen zurück an den Client.

Für Funktionen, die Echtzeitkommunikation erfordern, werden Websockets verwendet. Websockets ermöglichen eine bidirektionale Kommunikation zwischen Client und Server, die im Vergleich zu herkömmlichen HTTP-Anfragen effizienter und schneller ist.

4.3 Entwurf der Benutzeroberfläche

Die Ausgestaltung der Benutzeroberfläche erfolgte unter maßgeblicher Beteiligung des Autors. In der (Abbildung 5 Wireframe) ist ein Wireframe zu sehen, der einen Entwurf des Frontends darstellt. Der Wireframe präsentiert eine visuelle Darstellung der geplanten Benutzeroberfläche und dient der Planung sowie Überprüfung des Designs und der Funktionalität der Anwendung. Durch die Erstellung des Wireframes kann frühzeitig Feedback von Stakeholdern und Benutzern eingeholt werden, um das Design zu optimieren.

Die Abbildung 5 Wireframe präsentiert eine detaillierte Darstellung des Wireframes. Der Wireframe umfasst die einzelnen Interaktionen zur Navigation. Zusätzlich beinhaltet er die folgenden Hauptbereiche:

Login: Der Login-Bereich ermöglicht es Benutzern, sich mit ihren Zugangsdaten anzumelden. Das Design umfasst Eingabefelder für die E-Mail-Adresse und das Passwort sowie eine Schaltfläche zum Einloggen oder Registrieren.

Registrierung: Der Registrierungsbereich bietet neuen Benutzern die Möglichkeit, sich für die Nutzung der App zu registrieren. Der Wireframe zeigt Eingabefelder, für E-Mail-Adresse, Passwort und eine Bestätigung des Passworts sowie eine Schaltfläche zum Abschließen der Registrierung.

Homescreen: Diese Ansicht präsentiert eine Liste aller erstellten und erhaltenen Umfragen sowie eine Schaltfläche, über die eine neue Umfrage erstellt werden kann.

Umfrage erstellen: In diesem Bereich besteht für Benutzer die Möglichkeit, neue Umfragen zu erstellen. Der Wireframe beinhaltet Eingabefelder für die Eingabe des Umfragetitels, der Frage sowie der Antwortoptionen für die jeweiligen Fragetypen. Diese werden erst nach Festlegung des jeweiligen Fragetyps angezeigt. Die Erstellung der Umfrage erfolgt mittels einer Schaltfläche.

Umfrage teilen: Benutzer können ihre Umfragen über verschiedene Kanäle teilen. Der Wireframe zeigt die Optionen zum Teilen per Link.

Umfrage beantworten: In diesem Bereich können Benutzer an Umfragen teilnehmen und ihre Antworten einreichen. Der Wireframe zeigt die Umfrage mit den entsprechenden Antwortoptionen sowie eine Schaltfläche zum Einreichen der Antworten.

Umfrage kommentieren: Benutzer haben die Möglichkeit, Kommentare zu Umfragen abzugeben. Der Wireframe enthält ein Kommentarfeld und eine Schaltfläche zum Schreiben und Absenden des Kommentars.

Es sei darauf hingewiesen, dass das finale Design nicht zwangsläufig dem hier präsentierten Entwurf entsprechen muss. Die Rest-API wurde so konzipiert, dass das Frontend flexibel gestaltet werden kann.

4.4 Datenmodell

Die Erstellung der Datenbank erfolgte unter Berücksichtigung der Vorgaben des Lastenhefts.

Die Entitäten *Fragetyp*, *Frage*, *Umfrage*, *Antwort*, *Kommentar* und *User* bilden die Grundlage der Datenbank. Im *User* wird lediglich eine E-Mail gespeichert, was das einzige Attribut ist, über das eine Verbindung zur Person hergestellt werden kann. Dadurch soll die Erstellung von Fake-Accounts erschwert werden. Diese Vorgehensweise stellt eine ausgewogene Mischung aus Datenschutz und Funktionalität dar.

Die Beziehung zwischen *User* und *Umfrage* ist dadurch gekennzeichnet, dass ein *User* eine Umfrage erstellen kann, während er gleichzeitig *Umfragen* erhält, die er beantworten kann.

Innerhalb der *Umfrage* besteht zudem die Möglichkeit, ein Passwort zu hinterlegen, um lediglich autorisierte *User* an der Umfrage teilnehmen zu lassen.

In der Entität *Frage* muss die dynamische Länge von erstellten Fragen gespeichert werden.

Die Umsetzung erfolgte mittels JSON (JavaScript Object Notation) und auch in der *Antwort* mit der oder den gegebenen Antworten. Dies dient der Generierung der Anzeige im Frontend.

Die *Fragetyp*-Entität stellt eine Erweiterung des Funktionsumfangs der Applikation dar. Das Entity-Relationship-Diagramm (ERD) ist in [Abbildung 6 Entity Relationship Diagramm](#) zu finden, während das Datenbankmodell mit den Attributen in [Abbildung 7 Datenbankmodell](#) präsentiert wird.

4.5 Geschäftslogik

Das Geschäftsprozessdiagramm ([Abbildung 8 Geschäftsprozessdiagramm](#)) beinhaltet die User Storys aus [Anlage A 4.3 Funktionale Anforderungen](#)

Es wird davon ausgegangen, dass sich die User bereits authentifiziert haben. Das Diagramm veranschaulicht die Geschäftslogik der Anwendung und zeigt die Reihenfolge der Operationen und Entscheidungsprozesse. Im Folgenden werden einige zentrale Abläufe dargestellt:

Es besteht die Möglichkeit, eine Umfrage zu erstellen oder eine erhaltene zu beantworten. Bei der Erstellung einer Umfrage besteht die Möglichkeit, einen *Fragetyp* festzulegen (offen, Einzel- oder Mehrfachauswahl). Bei den Auswahlfeldern kann der Nutzer Antwortfelder festlegen. Bei einer offenen Umfrage besteht diese Möglichkeit nicht. Nach Erstellung der Umfrage wird im Frontend ein Link generiert, der geteilt werden kann.

Im Falle einer Umfrage zum Beantworten ist eine Einsicht in die Auswertung erst nach Beantwortung der Umfrage möglich. Dies dient der Sicherstellung einer unvoreingenommenen Beantwortung durch die Nutzer. Zudem besteht die Möglichkeit, die Umfrage zu kommentieren oder Kommentare zu lesen.

Der/die Umfrageersteller/in ist in der Lage, die Antworten ohne Einschränkungen einzusehen und die Umfrage zu löschen.

4.6 Festlegen der API-Endpunkte

Bezüglich der Festlegung der API-Endpunkte wurden zwei Varianten in Erwägung gezogen: die Festlegung pro Entität oder pro Screen, wie sie im Wireframe [Abbildung 5 Wireframe](#) definiert wurden. Letztlich wurde die erste Variante präferiert, da sie eine flexiblere Gestaltung des Frontends ermöglicht und keine Änderungen im Backend erforderlich macht, sofern sich lediglich das Frontend ändert, da alle Ressourcen erreichbar sind.

Das Schema der REST-API ist gemäß der Open-API-Spezifikation⁵ aufgebaut, wodurch eine einheitliche Struktur gewährleistet wird. Mithilfe von Swagger UI⁶ kann eine detaillierte Dokumentation der API erstellt werden. Zu Beginn soll die Implementierung aller HTTP-Methoden erfolgen, die zu einem späteren Zeitpunkt mit speziellen Pfaden erweitert werden können.

Des Weiteren soll die Möglichkeit der Pagination gegeben sein, wodurch die Anzahl der zu übertragenden Elemente festgelegt werden kann. Der Vorteil besteht darin, dass lediglich eine bestimmte Anzahl an Daten übermittelt wird und nicht der komplette Datensatz. Dies reduziert die Netzwerkbelastung.

4.7 Maßnahmen zur Qualitätssicherung

Die Maßnahmen zur Qualitätssicherung umfassen das Nutzen von Unit-Tests, um die einzelnen Endpunkte nach Änderungen am Code oder der Konfiguration zu testen. Für eine gleichbleibende Datenbank wurden Fixtures angelegt, welche Daten für die Datenbank erstellen.

Auf dieser Grundlage werden die Tests ausgeführt. Es wurde sich daran gehalten, erst die einzelnen Tests zu definieren, wobei die Vorgehensweise von TDD (Test-driven Development) zum Einsatz kommt. Vor der Implementierung einer Funktionalität wird ein Unit-Test geschrieben und ausgeführt. Im Falle eines fehlgeschlagenen Tests wird die betreffende Funktionalität implementiert und der Test erneut ausgeführt. Ist der Test erfolgreich durchgelaufen, wird der für die Funktionalität geschriebene Quellcode refaktoriert, um die Codequalität zu steigern.

Die Verwendung von Git sowie die Möglichkeit der Branches und Commits erlauben eine effiziente und nachvollziehbare Arbeitsweise. Einen Auszug finden Sie in *Abbildung 9 Versionskontrolle mit Sourcetree*.

4.8 Pflichtenheft

Nach Abschluss der Entwurfsphase wurde ein detailliertes Pflichtenheft erstellt, in dem die technischen Lösungen zur Erfüllung der Anforderungen des Lastenhefts dokumentiert sind. Es fungiert als technische Grundlage für die Implementierungsphase und gewährleistet, dass die implementierten Funktionen den Kundenerwartungen entsprechen. Ein Auszug des Pflichtenhefts ist in *Anlage 10 Pflichtenheft (Auszug) Seite ix* abgebildet.

5 Implementierungsphase

Die in Kapitel 4 (Entwurfsphase) erstellten Entwürfe bildeten die Grundlage für die anschließende Implementierung der einzelnen Elemente.

5.1 Implementierung der Datenstrukturen

Zur Erstellung der Datenbank wurde zunächst ein neues Projekt mithilfe der Symfony CLI (Command Line interface) angelegt und die erforderlichen Abhängigkeiten installiert (*Abbildung 10 Installierte Symfony Bundles*). Die Datenbank und ihre Entitäten werden mithilfe von Doctrine, einem ORM (Object-relational Mapping), erstellt. Dazu ist es erforderlich, in der `.env.test.local`-Datei die Datenbank-URL anzugeben (*Listing 1 Datenbank URL in .env.test.local*).

⁵ <https://spec.openapis.org/oas/latest.html>

⁶ <https://swagger.io/tools/swagger-ui/>

```
22  ##> doctrine/doctrine-bundle ##  
23  # Format described at https://www.doctrine-project.org/projects/doctrine-dbal/en/latest/reference/configuration.html  
24  # IMPORTANT: You MUST configure your server version, either here or in config/packages/doctrine.yaml  
25  #  
26  # DATABASE_URL="sqlite:///kernel.project_dir%/var/data.db"  
27  # DATABASE_URL="mysql://app:!ChangeMe!@127.0.0.1:3306/app?serverVersion=8.0.32&charset=utf8mb4"  
28  # DATABASE_URL="mysql://app:!ChangeMe!@127.0.0.1:3306/app?serverVersion=10.11.2-MariaDB&charset=utf8mb4"  
29  # DATABASE_URL="postgresql://app:!ChangeMe!@127.0.0.1:5432/app?serverVersion=16&charset=utf8"  
30  DATABASE_URL="mysql://root:@127.0.0.1:3306/listify_pro" #?serverVersion=8.0.32&charset=utf8mb4"  
31  
32  ##< doctrine/doctrine-bundle ##
```

Listing 1 Datenbank URL in `.env.test.local`

Im Anschluss können die einzelnen Entitäten und deren Beziehungen mithilfe von Konsolenbefehlen (`php bin/console make:entity`) erstellt werden. Das Bundle Doctrine generiert die einzelnen PHP-Entity-Klassen sowie die dazugehörigen Repository-Klassen, welche bereits die wichtigsten Datenbank Abfragen enthalten und in denen spezielle Datenbankabfragen formuliert werden können.

Es sei darauf hingewiesen, dass der *User* mit „`make:user`“ erstellt wird, wodurch sichergestellt wird, dass die erforderlichen Funktionen zur Authentifizierung enthalten sind.

Die Abbildung 1 Auszug Projektstruktur der Entitäten und Repositorys veranschaulicht die Struktur der einzelnen Klassen im Projekt.

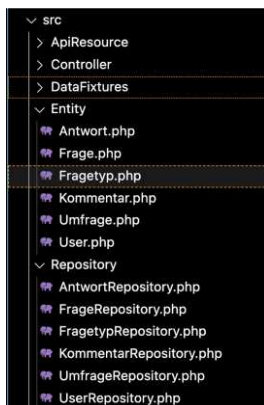


Abbildung 1 Auszug Projektstruktur der Entitäten und Repositorys

Nach dem Anlegen der Entitäten kann die Datenbank und das erstellte Schema erstellt werden. Die Datenbank wird mit weiteren Konsolenbefehlen erstellt (`php bin/console --env=test doctrine:database:create`). Anschließend kann das Schema erstellt werden (`php bin/console --env=test doctrine:schema:update --force`). Der Parameter „`env=test`“ weist darauf hin, dass die `.env.test.local`-Konfigurationen verwendet werden sollen.

Im Anschluss wurden die Fixtures erstellt, wie in Listing 2 Fixtures Funktionen (Auszug) dargestellt. Sie stellen die grundlegenden Daten in der Datenbank dar, anhand derer die Endpunkte getestet werden können.

Um die Fixtures zu laden, wird der folgende Befehl verwendet (Listing 3 Befehl zum Laden der Fixtures in die Datenbank). Dieser stellt sicher, dass die Auto-Inkrement-IDs beim erneuten Laden wieder bei 1 beginnen. Diese Tatsache ist von entscheidender Bedeutung, da sich einzelne Sequenzen in den Funktionen zum Erstellen der Fixtures auf die IDs beziehen.

5.2 Implementierung der Geschäftslogik

Die Implementierung der REST API erforderte zunächst die Definition von Testfällen. Exemplarisch sei hier die Testklasse für die User-Authentifizierung genannt (Listing 4 Testklasse User Registration).

Die Entitätsklassen wurden mithilfe von API-Plattform⁷ als API-Ressourcen deklariert. Dies erfolgte über Annotationen. Zudem wurden Validierungen und erweiterte Routen festgelegt sowie Filter eingerichtet. Um spezifische Daten an das Frontend zu senden, um dieses zu entlasten, wurde nach erfolgreichem Absolvieren der Tests der einzelnen Entitäten noch die Authentifizierung hinzugefügt. Es wurde sich für JWT (JSON Web Token) entschieden, da dieser durch seinen geringen Netzwerk-Overhead insbesondere bei mobilen Anwendungen von Vorteil ist. Über ihn kann auch die TTL (Time to Live) in der Konfiguration festgelegt werden. Des Weiteren wurden die Tests erweitert, sodass sie auch mit dem JWT funktionieren. Dieses fungiert als Schlüssel für jeden Client, um Zugriff auf die Endpunkte zu erhalten. Dazu wurde mit Hilfe des Symfony-Bundels „lexik/jwt-authentication-bundle“ diese Funktion über Konfigurationsdateien ermöglicht (Listing 5 Konfiguration JWT).

Im Rahmen dessen war eine Modifikation erforderlich, da die Funktionalität nicht wie in der offiziellen Dokumentation⁸ beschrieben gegeben war.

Des Weiteren ergaben sich Schwierigkeiten bei der Registrierung eines neuen Users. Auch hier war es nicht möglich, die Registrierung wie in der offiziellen Dokumentation⁹ beschreiben durchzuführen. Daher wurde ein eigener Controller (Listing 6 Controller zur Registrierung) zur Registrierung und zum Hashen des Passwortes erstellt, da die Annotations der API-Plattform umgangen werden müssen, was eine zusätzliche Validierung erforderlich machte. Diese spezielle Route (`/api/registerUser`) wird in der API-Dokumentation (Abbildung 11 Auszug 1 API-Dokumentation) nicht berücksichtigt. Sie erwartet ein JSON-Request in der Form von Abbildung 2 JSON Request. Die Werte der Attribute sind an dieser Stelle exemplarisch eingetragen.

```
1  {  
2    "email": "beispiel@ars1992.de",  
3    "password": "superGeheim",  
4    "name": "Alessandro"  
5  }
```

Abbildung 2 JSON Request

Aufgrund der genannten Komplikationen wurde kein WebSocket für die Kommentarfunktion implementiert. Dies beeinträchtigt jedoch nicht die Gesamtfunktionalität. Der Client ist dennoch in der Lage, die Kommentare abzurufen.

6 Abnahmephase

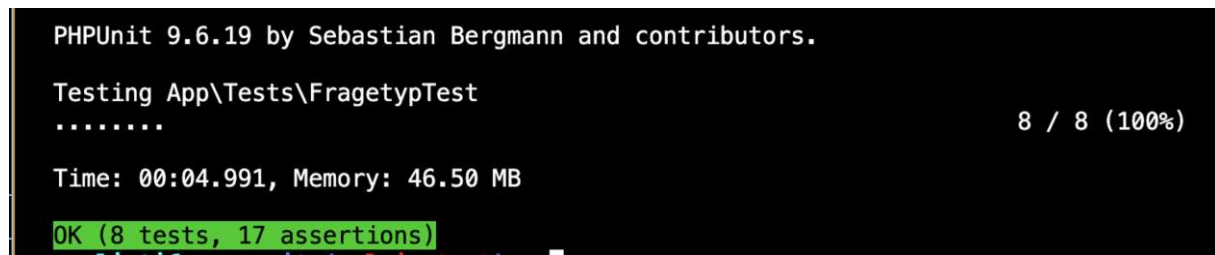
In der Abnahmephase wurden diverse Maßnahmen ergriffen, um die Qualität und Funktionalität der Anwendung sicherzustellen. Zu diesen Maßnahmen zählten Unit-Tests, welche bereits während der Entwicklung regelmäßig durchgeführt wurden, sowie Code-Reviews.

⁷ <https://api-platform.com/>

⁸ <https://api-platform.com/docs/core/jwt/>

⁹ <https://api-platform.com/docs/core/user/>

Schließlich wurden alle Tests noch einmal durchgeführt und ein abschließendes Code-Review durchgeführt. Der bestandene Test ist in [Abbildung 3 Testergebnis der Entität Fragetyp](#) dargestellt und betrifft die Entität „Fragetyp“.



```
PHPUnit 9.6.19 by Sebastian Bergmann and contributors.  
  
Testing App\Tests\FragetypTest  
..... 8 / 8 (100%)  
  
Time: 00:04.991, Memory: 46.50 MB  
  
OK (8 tests, 17 assertions)
```

Abbildung 3 Testergebnis der Entität Fragetyp

7 Dokumentation

Die Bereitstellung einer API-Dokumentation aller Endpunkte erfolgt mittels der API-Plattform. Die Veranschaulichung erfolgt in [Abbildung 11 Auszug 1 API-Dokumentation](#) und [Abbildung 12 Auszug 2 API-Dokumentation](#). Die Dokumentation ermöglicht zudem das Ausprobieren der Endpunkte, wodurch es für das Frontendteam leichter ersichtlich wird, welche Ressourcen zur Verfügung stehen.

Aufgrund der noch nicht abgeschlossenen Entwicklung des Frontends wurde ein Anwenderdokumentation auf Basis der Wireframes erstellt. Dieses beschreibt die einzelnen Interaktionen.

Diese Informationen finden Sie im Anhang 17 Anwenderdokumentation Seite xvii.

8 Fazit

Im Rahmen des Projektabschlusses wurde eine retrospektive Betrachtung des Projekts vorgenommen, um ein Fazit zu ziehen. Dazu wurde ein Soll-Ist-Vergleich durchgeführt ([Tabelle 4 Soll-Ist-Vergleich](#)). Im Anschluss wurden die gesammelten Erfahrungen und Ergebnisse zusammengefasst, um schließlich einen Blick in die strahlende Zukunft dieses Projekts zu richten.

8.1 Soll-/Ist-Vergleich

Der Vergleich zeigt die deutlichsten Abweichungen ([Tabelle 4 Soll-Ist-Vergleich](#)) während der Analysephase (+4h) und der Implementierungsphase (-6h). Die selbständige Fortbildung war sehr hilfreich, um die richtigen Symfony-Bundels zu wählen. Die Implementierung der API-Endpunkte konnte folglich komfortabel umgesetzt werden. Bei der Wirtschaftlichkeitsanalyse stellte sich das Problem, mögliche Zukunftsszenarien zu finden, da die App erstmals kostenfrei sein wird. Durch die Probleme bei der Implementierung der User-Entität und der dahingehenden Implementierung ohne API-Plattform konnte die angegebene Zeit zum Debugging nicht erfüllt werden. Dies resultierte auch in einem erhöhten Dokumentationsaufwand.

Die API ist funktional und das Frontend kann so umgesetzt werden. Allerdings wird im Moment noch die Auswertung der Umfragen auf dem Client ausgelagert, was suboptimal ist. Die Nichtimplementierung des Websockets zur Echtzeitkommunikation fällt dahingehend nicht so ins Gewicht, könnte aber auch nicht in diesem Zeitrahmen umgesetzt werden.

8.2 Lessons Learned

Im Rahmen der Projektdurchführung konnten mehrere wichtige Erkenntnisse gewonnen werden, die im Folgenden zusammengefasst werden sollen:

Eine der größten Herausforderungen stellte die teilweise unzureichende Dokumentation der verwendeten Bibliotheken dar. Es konnte festgestellt werden, dass die Dokumentation möglicherweise nicht auf dem neuesten Stand ist. Dennoch sind diese Dokumentationen von großem Wert, da sie grundlegende Informationen und Beispiele bieten, die als Ausgangspunkt dienen können.

Die Entwurfsphase erwies sich als essenziell für den Erfolg des Projekts. Ein sorgfältiger Entwurf hilft, spätere Probleme zu vermeiden und die Implementierungsphase erheblich zu erleichtern. Es zeigte sich, dass eine gute Planung und ein durchdachtes Design wesentlich dazu beitragen, die Entwicklung effizient und zielgerichtet zu gestalten.

Ein weiterer wichtiger Punkt war die Zeitplanung. Ursprünglich wurde ein größerer Zeitblock von 30 Stunden geplant. Diese Vorgehensweise erwies sich als zu groß und unflexibel, da es schwierig war, den Fortschritt genau zu überwachen und anzupassen. Kleinere Zeitblöcke wären hilfreicher gewesen, um den Fortschritt besser zu kontrollieren und flexibler auf unvorhergesehene Ereignisse reagieren zu können.

Diese Erkenntnisse werden in zukünftigen Projekten berücksichtigt, um die Planung und Durchführung weiter zu optimieren.

8.3 Ausblick

Der Ausblick auf das Projekt ist positiv und zeigt mehrere geplante Erweiterungen und Verbesserungen.

Der nächste Schritt im Projekt ist die Fertigstellung und Integration des Frontends. Dies wird es den Nutzern ermöglichen, auf die Funktionen des Systems über eine benutzerfreundliche Oberfläche zuzugreifen und die Interaktionen zu ermöglichen. Nach der Integration des Frontends wird das Projekt veröffentlicht. Im Anschluss daran sind kontinuierliche Erweiterungen geplant, um zusätzliche Fragetypen zu integrieren. Die neuen Fragetypen werden zunächst exklusiv im Pro-Abonnement verfügbar sein, was einen Anreiz zur Nutzung der erweiterten Version schafft und zusätzliche Einnahmequellen eröffnet.

Um die Clients zu entlasten und die Performance zu verbessern, wird die API weiterentwickelt. Diese Erweiterung zielt darauf ab, die Kommunikation zwischen Server und Clients effizienter zu gestalten und die Last auf den Clients zu minimieren. Des Weiteren ist die Implementierung des noch nicht umgesetzten Websockets für die Kommentarfunktion vorgesehen.

Nach dem Deployment besteht die Möglichkeit, auf Erfahrungen und Wünsche der Nutzer einzugehen, um weitere Funktionen zu implementieren.

Die geplanten Maßnahmen zielen darauf ab, das Projekt nicht nur nutzerfreundlicher und leistungsfähiger zu gestalten, sondern auch eine solide Grundlage für zukünftige Entwicklungen und Erweiterungen zu schaffen.

Anhang

Anlage 1 Detaillierte Zeitplanung

Projektphase	Geplante Zeit
Analyse	12 h
Selbständige Fortbildung	7 h
Absprache mit dem Projektleiter	2 h
Ist-Analyse	1 h
Wirtschaftlichkeitsanalyse	2 h
Entwurf	15 h
Erstellung eines ERM und Datenbankmodels	3 h
Erstellung des Pflichtenhefts	4 h
Erstellung eines Geschäftsprozessdiagramm	4 h
Festlegen der API-Endpunkte	4 h
Implementierung inkl. Tests	41 h
Erstellung der Datenbank	3 h
Implementierung der Backend-Funktionen	30 h
Testphase	5 h
Debugging	3 h
Abnahme	2 h
Qualitätssicherung	2 h
Dokumentation	10 h
Anwenderdokumentation	2 h
Technische Dokumentation	8 h
Gesamt	80 h

Tabelle 3 Detaillierte Zeitplanung

Anlage 2 Zeitplanung in Odoo

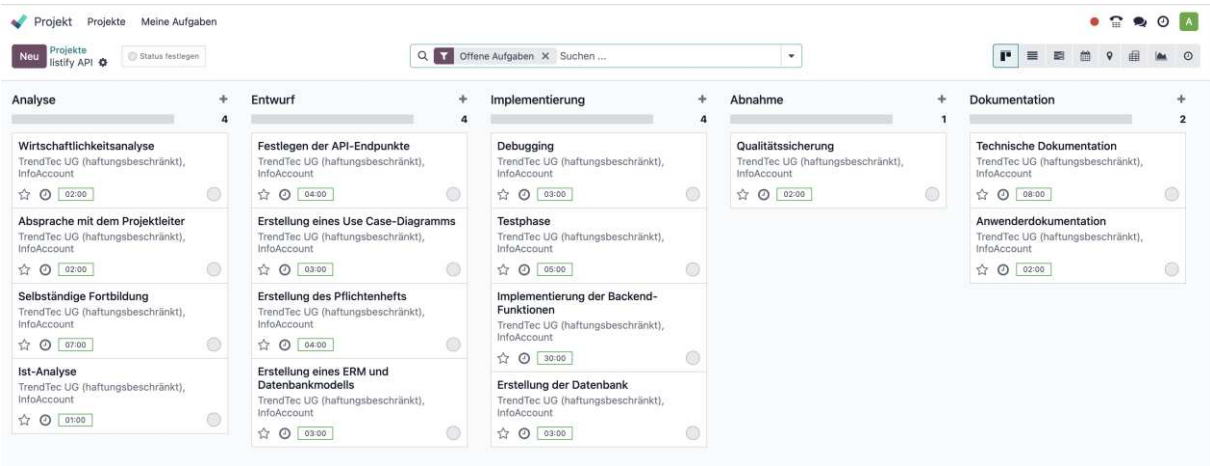


Abbildung 4 Zeitplanung in Odoo

Anlage 3 Verwendete Ressourcen

Räumlichkeiten

- Büro

Hardware

- Laptop
- Desktop PC
- Zwei 24-Zoll-Monitore

Software

- **Draw.io**¹⁰: Programm zur Erstellung von Diagrammen und Modellen
- **Git**¹¹ und **GitHub**¹²: Für Versionskontrolle und kollaborative Entwicklung. GitHub dient als Plattform zur gemeinsamen Arbeit an Projekten, zur Verfolgung von Änderungen und zur Verwaltung von Code-Repositories.
- **Sourcetree**¹³: Verwaltung der Git Versionskontrolle.
- **Visual Studio Code (VSC)**¹⁴: Als integrierte Entwicklungsumgebung (IDE) für die Softwareentwicklung.
- **XAMPP**¹⁵: Als Entwicklungsumgebung, um einen lokalen Webserver mit Apache, MySQL und PHP einzurichten.
- **MySQL**¹⁶: Als relationales Datenbankmanagementsystem (RDBMS) für die Speicherung und Verwaltung von Daten.
- **Odoo**¹⁷: Für Projektmanagement und Zeitüberwachung

Personal

- **Auszubildender/Umschüler Fachinformatiker für Anwendungsentwicklung**: Verantwortlich für die Umsetzung des Projekts.
- **Praktikanten**: Unterstützen bei der Durchführung von Reviews.
- **Projektleiter**: Unterstützung bei Problemstellungen und Bereitstellung der benötigten Ressourcen sowie die Koordination und Überwachung des gesamten Projekts.
- **Frontendteam**: Entwickeln die Benutzeroberfläche der Umfrage-App und sorgen für die Integration mit dem Backend.

¹⁰ <https://app.diagrams.net/>

¹¹ <https://git-scm.com/>

¹² <https://github.com/>

¹³ <https://www.sourcetreeapp.com/>

¹⁴ <https://code.visualstudio.com/>

¹⁵ <https://www.apachefriends.org/de/index.html>

¹⁶ <https://www.mysql.com/de/>

¹⁷ https://www.odoo.com/de_DE

Anlage 4 Lastenheft (Auszug)

A 4.1 Beschreibung des Projekts:

Die Umfrage-PWA ermöglicht Benutzern die Erstellung, Verwaltung und Durchführung von Umfragen. Sie bietet eine intuitive Benutzeroberfläche und unterstützt Funktionen wie das Teilen von Umfragen, die Erfassung von Antworten in Echtzeit, detaillierte Analysen und Berichte sowie eine Kommentarfunktion.

A 4.2 Umfang:

Der Umfang des Projekts umfasst die Entwicklung einer voll funktionsfähigen PWA, die folgende Hauptfunktionen unterstützt:

- Benutzerregistrierung und Authentifizierung
- Erstellung und Verwaltung von Umfragen mit verschiedenen Antwortoptionen
- Teilen von Umfragen über Links.
- Erfassung von Antworten in Echtzeit und Speicherung in einer Datenbank
- Generierung von detaillierten Berichten und Analysen basierend auf den gesammelten Antworten
- Unterstützung für verschiedene Geräte und Browser durch eine responsive Benutzeroberfläche
- Kommentarfunktion zu Umfragen.

A 4.3 Funktionale Anforderungen

Als Umfrageersteller:

möchte ich eine neue Umfrage erstellen können, damit ich Feedback von meinen Kontakten sammeln und Meinungen zu bestimmten Themen einholen kann.

Als Umfrageersteller:

möchte ich Fragen mit verschiedenen Typen wie Text, Mehrfachauswahl, Einzelauswahl usw. hinzufügen können, damit ich meine Umfrage anpassen und vielfältige Feedbackoptionen bereitstellen kann.

Als Umfrageersteller:

möchte ich die Möglichkeit haben, Fragen und Antwortoptionen in der erstellten Umfrage zu bearbeiten oder zu löschen, damit ich Änderungen vornehmen und die Umfrage flexibel anpassen kann.

Als Umfrageersteller:

möchte ich die Umfrage über einen einzigartigen Link teilen können, damit ich sie mit meinen Kontakten über soziale Medien, Messaging-Apps oder E-Mail teilen und eine größere Teilnehmerzahl erreichen kann.

Als Umfrageersteller:

möchte ich die Datenschutzeinstellungen für die Umfrage anpassen und kontrollieren können, damit ich die Privatsphäre und den Zugriff auf die Umfrage steuern und sicherstellen kann, dass nur autorisierte Personen darauf zugreifen können.

Als Umfrage-Teilnehmer:

möchte ich in Echtzeit die Antworten auf die Umfrage sehen können, damit ich die aktuellen Ergebnisse verfolgen und möglicherweise meine eigenen Antworten basierend auf anderen Antworten anpassen kann.

Als Umfrage-Teilnehmer:

möchte ich zu einer Umfrage Kommentare hinzufügen können, damit ich meine Gedanken, Meinungen oder weitere Details zu den gestellten Fragen teilen und mit anderen Teilnehmern interagieren kann.

Als Umfrage-Teilnehmer:

möchte ich die Kommentare anderer Teilnehmer lesen und darauf antworten können, damit ich an Diskussionen teilnehmen und verschiedene Perspektiven zu den gestellten Fragen kennenlernen kann.

A 4.4 Nicht Funktionale Anforderungen

Leistung:

Die Umfrage-PWA muss eine hohe Leistung bieten, um eine reibungslose Benutzererfahrung zu gewährleisten.

Spezifikation:

Die Ladezeit der Umfrage-PWA sollte unter 3 Sekunden liegen, um eine schnelle Navigation zu ermöglichen.

Sicherheit:

Die Sicherheit der Benutzerdaten und der Umfrageinhalte ist von höchster Priorität.

Spezifikation:

Die Umfrage-PWA muss SSL/TLS-Verschlüsselung verwenden, um die Datenübertragung zwischen Benutzergeräten und dem Server zu sichern.

Skalierbarkeit:

Die Umfrage-PWA sollte skalierbar sein, um ein zunehmendes Benutzerwachstum zu unterstützen.

Spezifikation:

Die Umfrage-PWA muss in der Lage sein, bis zu 10.000 gleichzeitige Benutzeranfragen zu verarbeiten, ohne dass die Leistung beeinträchtigt wird.

Benutzerfreundlichkeit:

Die Benutzeroberfläche der Umfrage-PWA sollte intuitiv und benutzerfreundlich sein, um eine einfache Navigation und Interaktion zu ermöglichen.

Spezifikation:

Die Umfrage-PWA muss barrierefrei sein und die Richtlinien für Web Content Accessibility Guidelines (WCAG) erfüllen, um eine optimale Zugänglichkeit für alle Benutzer sicherzustellen.

Responsives Design:

Die Umfrage-PWA muss ein responsives Design haben, das sich an verschiedene Bildschirmgrößen und Geräte anpasst.

Spezifikation:

Die Umfrage-PWA muss auf mobilen Geräten, Tablets und Desktops gleichermaßen gut funktionieren und sich dynamisch an verschiedene Bildschirmgrößen anpassen.

Browserkompatibilität:

Die Umfrage-PWA muss in verschiedenen Webbrowsern konsistent funktionieren.

Spezifikation:

Die Umfrage-PWA muss in den neuesten Versionen der gängigen Webbrowser wie Google Chrome, Mozilla Firefox, Safari und Microsoft Edge vollständig unterstützt werden.

Anlage 5 Benutzeroberfläche

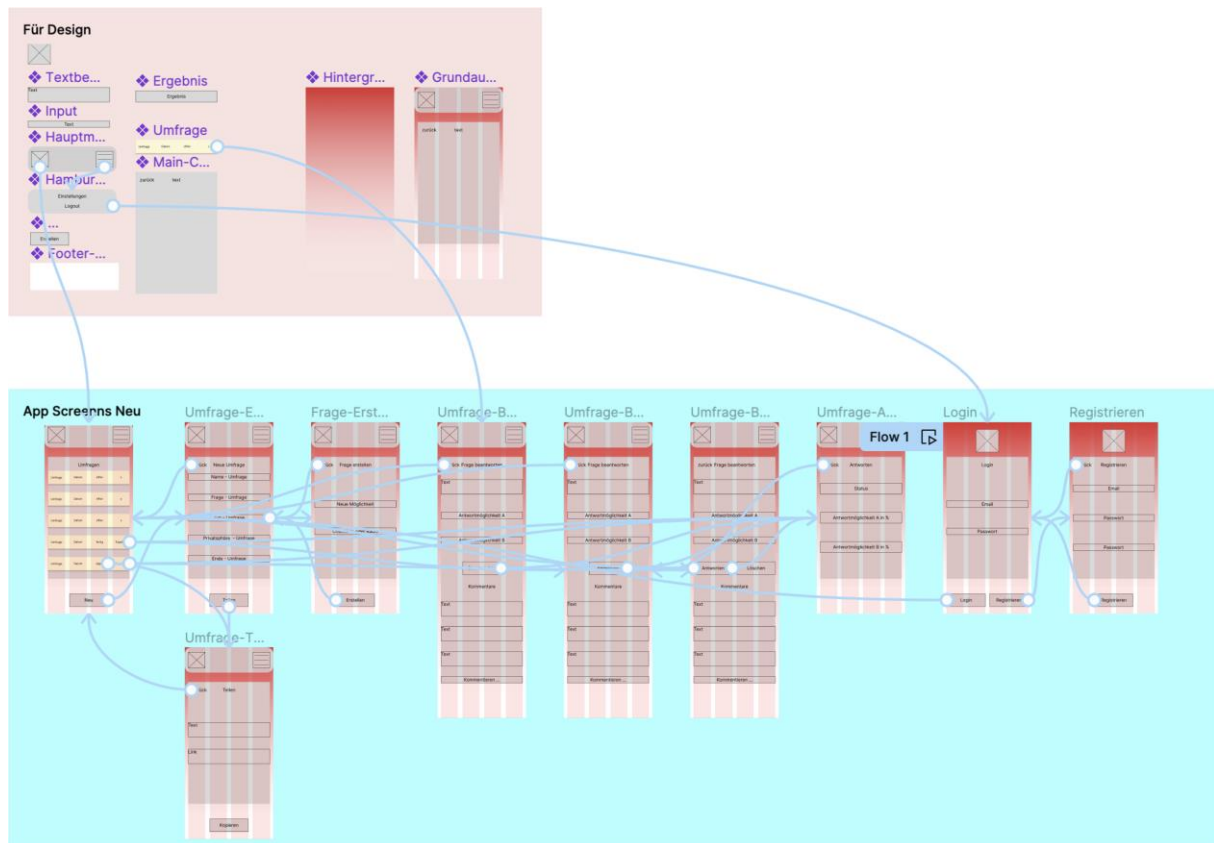


Abbildung 5 Wireframe

Anlage 6 Entity Relationship Diagramm

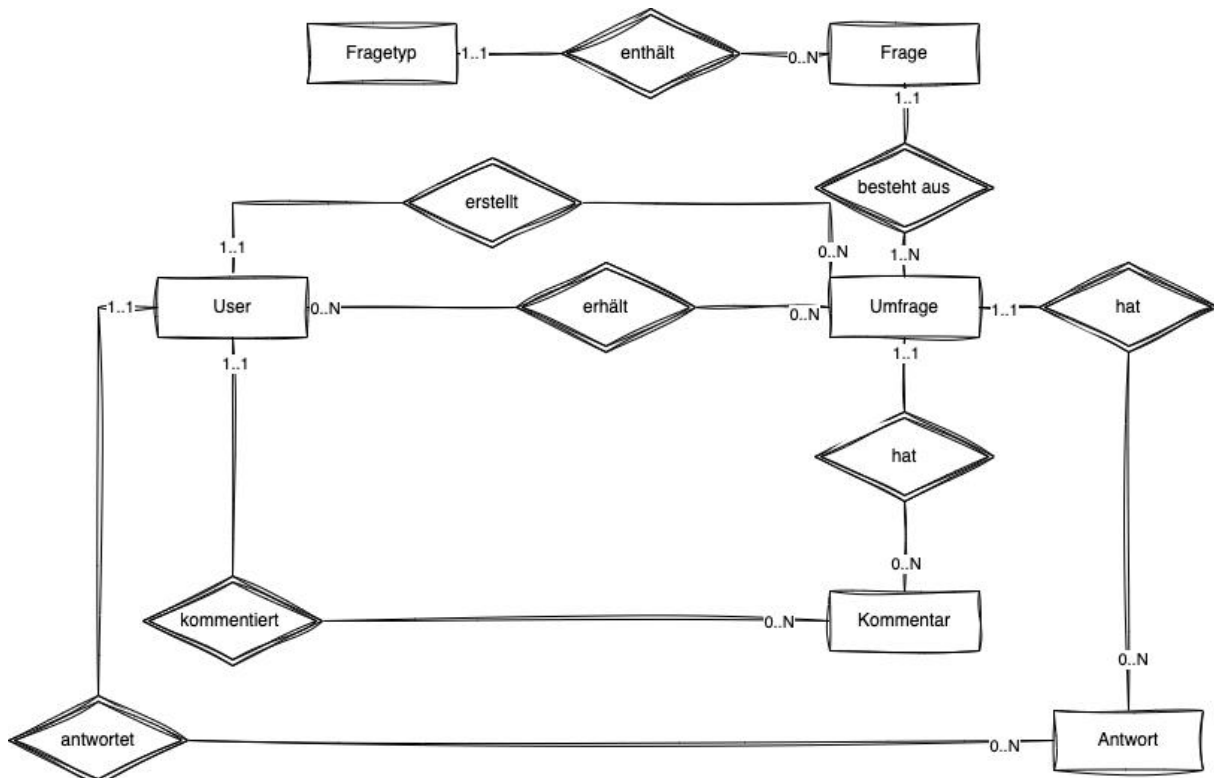


Abbildung 6 Entity Relationship Diagramm

Anlage 7 Datenbankmodell

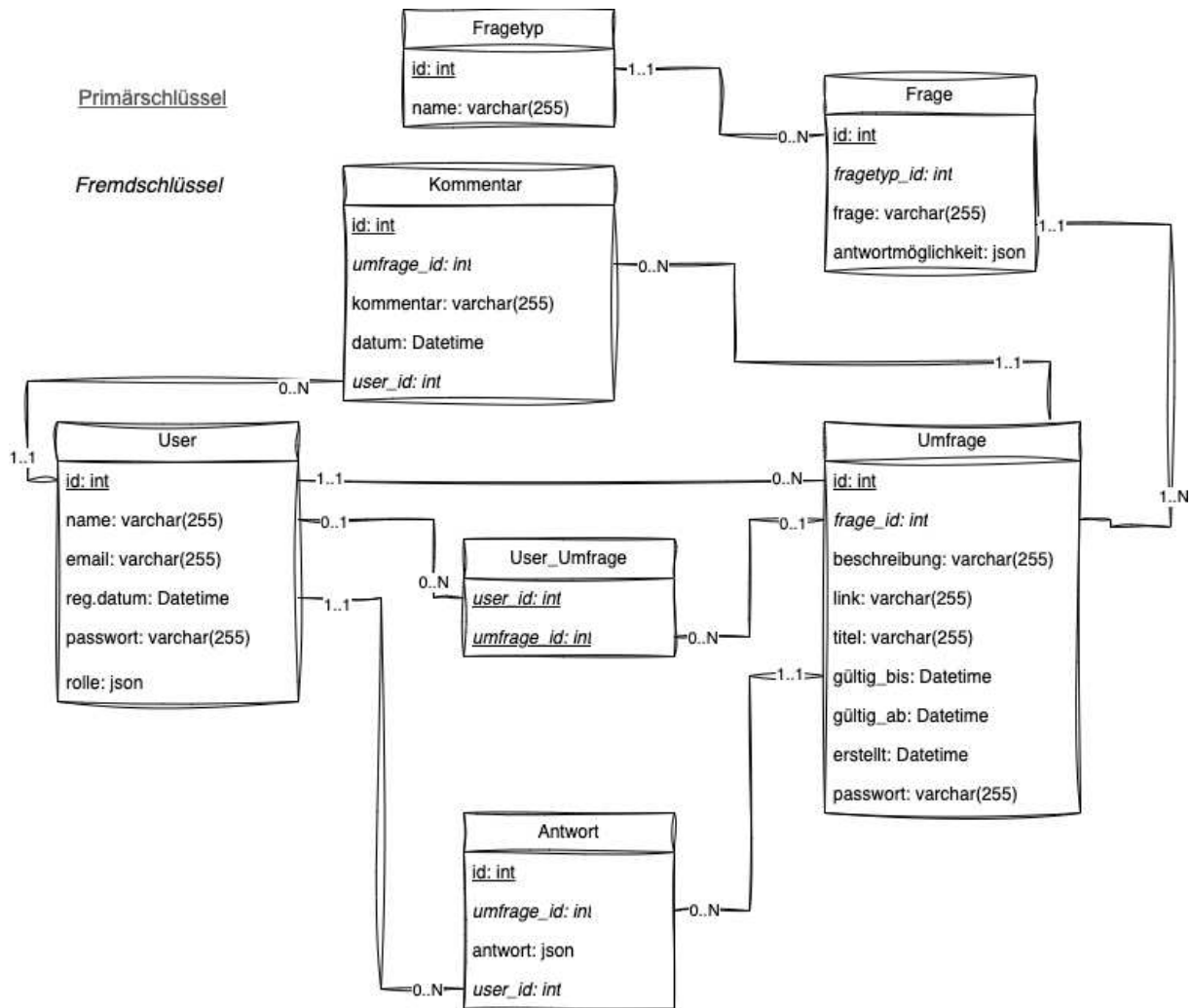


Abbildung 7 Datenbankmodell

Anlage 8 Geschäftsprozessdiagramm

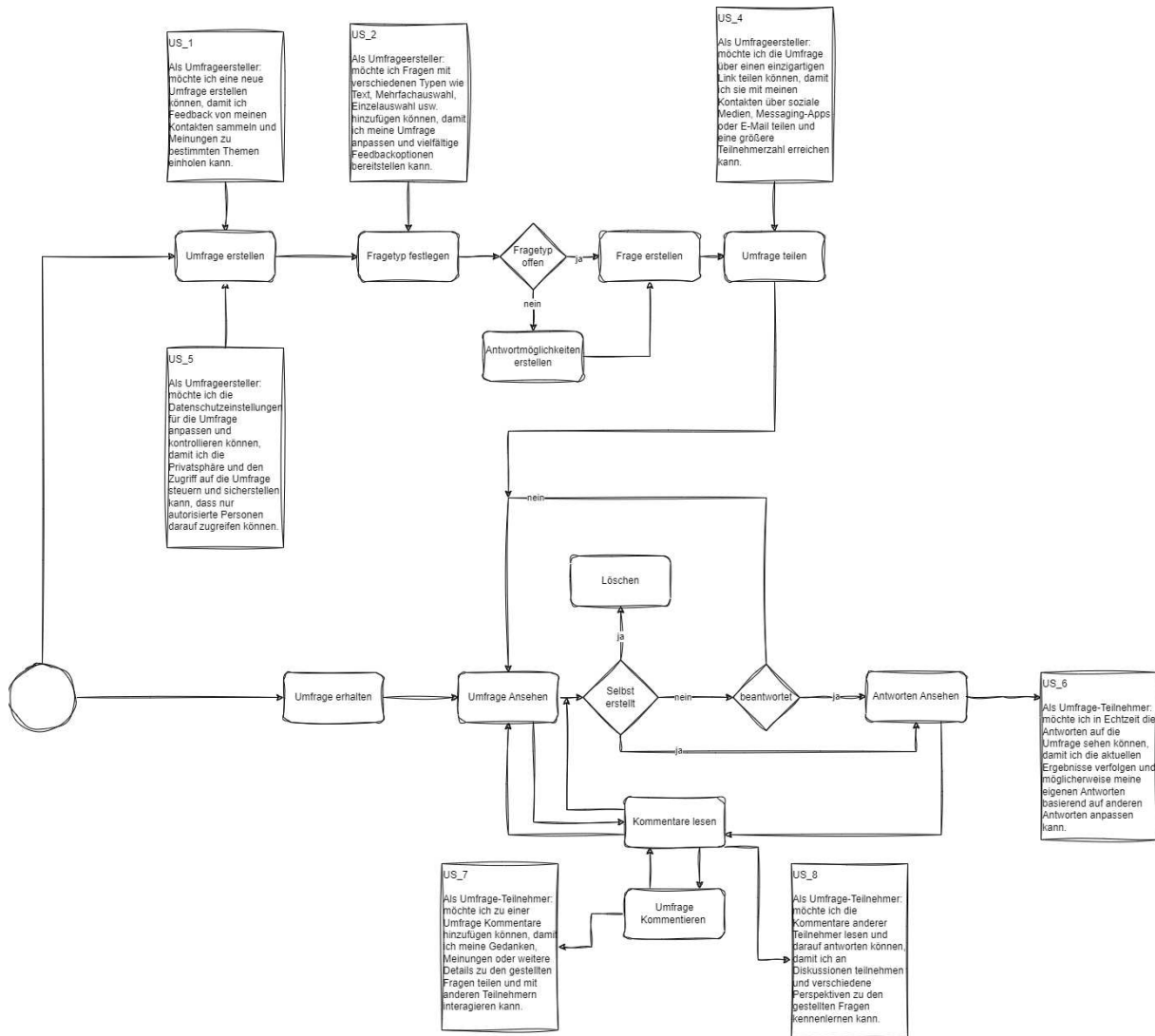


Abbildung 8 Geschäftsprozessdiagramm

Anlage 9 Versionskontrolle

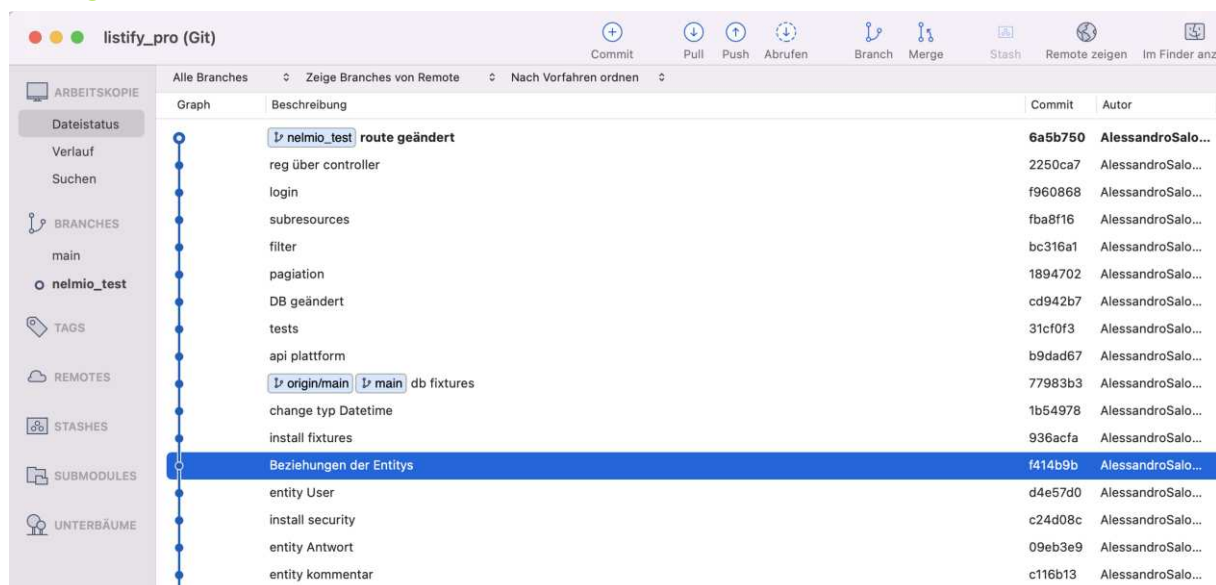


Abbildung 9 Versionskontrolle mit Sourcetree

Anlage 10 Pflichtenheft (Auszug)

A 10.1 Betriebsbedingungen

Die Umfrage-APP ist darauf ausgelegt, unter folgenden Betriebsbedingungen effizient zu funktionieren.

Internetverbindung: Die Software erfordert eine stabile Internetverbindung für den Zugriff auf die Umfragen, das Teilen von Umfragen, die Erfassung von Antworten in Echtzeit und die Interaktion mit anderen Benutzern über die Kommentarfunktion.

Gerätekompatibilität: Die Umfrage-App ist sowohl für den Einsatz auf mobilen Geräten wie Smartphones und Tablets konzipiert. Es wird erwartet, dass sie auf gängigen Betriebssystemen wie iOS und Android problemlos funktioniert.

Datenschutz und Sicherheit: Es wird erwartet, dass die Umfrage-App alle geltenden Datenschutzbestimmungen einhält und Sicherheitsmaßnahmen implementiert, um die Integrität und Vertraulichkeit der Benutzerdaten zu gewährleisten. Dabei muss der Datenschutz dem Stand der Technik entsprechen und Maßnahmen zur Datenminimierung beinhalten.

Wartung und Aktualisierungen: Die Software erfordert regelmäßige Wartung und Aktualisierungen, um eine optimale Leistung und Sicherheit zu gewährleisten. Dies wird vom Entwicklerteam übernommen. Die Umfrage-App sollte unter diesen Betriebsbedingungen reibungslos funktionieren und eine effiziente Durchführung von Umfragen sowie eine reibungslose Interaktion mit den Benutzern ermöglichen.

A 10.2 Änderung zum Lastenheft

Ursprünglich wurde die Umsetzung einer Progressive Web App (PWA) gemäß den Anforderungen im Lastenheft geplant. Allerdings gab es anfängliche Bedenken hinsichtlich der Einschränkungen seitens Apple, die die Nutzung von PWAs betrafen. Diese Bedenken sind mittlerweile größtenteils ausgeräumt, aber es besteht noch eine gewisse Unsicherheit bezüglich der genauen Umsetzungsmöglichkeiten unter den aktuellen Richtlinien von Apple.

Grund für die Änderung: Die Entscheidung von Apple, Maßnahmen zu ergreifen, die die Nutzung von PWAs auf ihren Geräten einschränken, zwingt uns dazu, unsere Implementierungsstrategie zu überdenken. PWAs wurden ursprünglich als Kernfunktionalität geplant, um eine plattformübergreifende Benutzererfahrung zu ermöglichen. Jedoch macht die beschränkte Unterstützung von PWAs seitens Apple eine Umsetzung gemäß den ursprünglichen Anforderungen unmöglich.

Beschreibung der Änderung: Aufgrund der Einschränkungen bei der Implementierung einer reinen PWA auf iOS-Geräten, wird stattdessen eine hybride Lösung verfolgt. Wir werden eine App entwickeln, die plattformübergreifend auf iOS und Android funktioniert, unter Verwendung von React Native. Diese hybride Lösung ermöglicht es, die wesentlichen Funktionen der ursprünglich geplanten PWA zu replizieren und gleichzeitig eine optimierte Benutzererfahrung auf beiden Plattformen zu gewährleisten. React Native bietet eine effiziente Möglichkeit, native Funktionen in die App zu integrieren und die spezifischen Anforderungen von iOS und Android zu erfüllen. Indem wir React Native nutzen, können wir Ressourcen effizienter einsetzen und Entwicklungszeit sparen, da der Großteil des Codes plattformübergreifend verwendet werden kann. Dies ermöglicht es uns, die Änderung mit minimalen Auswirkungen auf den Zeitplan und das Budget des Projekts umzusetzen.

Auswirkungen der Änderung: Die Entscheidung, die App mit React Native für iOS und Android zu entwickeln, sowie zusätzlich eine Webseite zu erstellen, hat Auswirkungen auf den Entwicklungszeitplan und das Budget des Projekts. Es ist wichtig zu betonen, dass sich diese Änderung nur auf die Frontend-Entwicklung bezieht und keine Auswirkungen auf das Backend des Systems hat. Das Backend bleibt unverändert und wird weiterhin die erforderlichen Funktionen für sowohl die App als auch die Webseite bereitstellen. Die Erstellung einer Webseite erfordert zusätzliche Ressourcen und Zeit. Während einige Elemente möglicherweise von der App übernommen werden können, müssen bestimmte Aspekte speziell für die Webumgebung angepasst werden, um eine optimale Benutzererfahrung zu gewährleisten.

Risiken und Chancen: Die Hauptgefahr besteht darin, dass die zusätzliche Komplexität der hybriden Lösung zu Verzögerungen und unvorhergesehenen Herausforderungen führen könnte. Auf der positiven Seite bietet die hybride Lösung die Möglichkeit, eine nahtlose Benutzererfahrung auf allen Plattformen zu gewährleisten und die Anforderungen von Apple zu erfüllen.

A 10.3 Produktfunktionen

Die Umfrage-App bietet eine Vielzahl von Funktionen, um den Anforderungen sowohl der Umfrageersteller als auch der Umfrage-Teilnehmer gerecht zu werden. Zu den Hauptfunktionen gehören:

Umfrageerstellung: Umfrageersteller können neue Umfragen erstellen, um Feedback von ihren Kontakten zu sammeln und Meinungen zu verschiedenen Themen einzuholen. Dabei können sie Fragen mit verschiedenen Typen wie Text, Mehrfachauswahl, Einzelauswahl usw. hinzufügen, um eine Vielzahl von Feedbackoptionen bereitzustellen.

Bearbeitung und Löschung von Kommentaren: Umfrageersteller und Teilnehmer haben die Möglichkeit, Kommentare zu Löschen

Teilen von Umfragen: Umfrageersteller können die Umfrage über einen einzigartigen Link teilen, um sie mit ihren Kontakten über soziale Medien, Messaging-Apps oder E-Mail zu teilen und eine größere Teilnehmerzahl zu erreichen.

Datenschutzeinstellungen: Umfrageersteller können die Datenschutzeinstellungen für die Umfrage anpassen und kontrollieren, um die Privatsphäre und den Zugriff auf die Umfrage zu steuern und sicherzustellen, dass nur autorisierte Personen darauf zugreifen können.

Echtzeitantworten: Umfrage-Teilnehmer können in Echtzeit die Antworten auf die Umfrage sehen, um die aktuellen Ergebnisse zu verfolgen.

Kommentarfunktion: Umfrage-Teilnehmer haben die Möglichkeit, zu einer Umfrage Kommentare hinzuzufügen, um ihre Gedanken, Meinungen oder weitere Details zu den gestellten Fragen zu teilen und mit anderen Teilnehmern zu interagieren.

A 10.4 Daten

Beschreibung der zu verarbeitenden Daten: Die Umfrage-App verarbeitet verschiedene Arten von Daten, die für die Erstellung, Verwaltung und Durchführung von Umfragen erforderlich sind. Dazu gehören: Benutzerdaten wie Name, E-Mail-Adresse und Profilinformationen für die Registrierung und Authentifizierung von Benutzern. Umfragedaten wie Fragen, Antwortoptionen, Umfrageergebnisse und Kommentare. Metadaten wie Zeitstempel, Benutzeraktionen und Umfragestatistiken zur Analyse und Berichterstellung.

A 10.5 Datenhaltung: Die Datenhaltung erfolgt in einer sicheren und skalierbaren Datenbank, die speziell für die Umfrage-App entwickelt wurde. Als DBMS wird MySQL verwendet. Die Datenbank ermöglicht die effiziente Speicherung und Verwaltung von Benutzerdaten, Umfragedaten und Metadaten. Sie bietet Funktionen wie Datenintegrität, Sicherheit, Backup und Wiederherstellung sowie Skalierbarkeit, um das wachsende Datenverarbeitungsvolumen zu bewältigen.

A 10.6 API-Entwicklung

Die API für die Umfrage-App wird unter Verwendung von Symfony 7.0 entwickelt. Symfony ist ein bewährtes PHP-Framework, das eine robuste und skalierbare Grundlage für die Entwicklung von Webanwendungen bietet. Mit Symfony können wir eine leistungsstarke und gut strukturierte API erstellen, die die Interaktion zwischen der App und der Datenbank effizient verwaltet.

Die Verwendung von Symfony 7.0 ermöglicht es uns, bewährte Best Practices in der API-Entwicklung zu implementieren und gleichzeitig eine hohe Codequalität und Wartbarkeit sicherzustellen.

A 10.7 Entwicklungsumgebung

Auf Auftragnehmer Seite verwenden wir für die Konzeption der Lösung folgende Werkzeuge und technische Basis:

Visual Studio Code (VSC) als integrierte Entwicklungsumgebung (IDE) für die Softwareentwicklung.

XAMPP als Entwicklungsumgebung, um einen lokalen Webserver mit Apache, MySQL und PHP einzurichten.

MySQL als relationales Datenbankmanagementsystem (RDBMS) für die Speicherung und Verwaltung von Daten.

Git und GitHub für Versionskontrolle und kollaborative Entwicklung. GitHub dient als Plattform zur gemeinsamen Arbeit an Projekten, zur Verfolgung von Änderungen und zur Verwaltung von Code-Repositories.

Diese Werkzeuge und technische Basis ermöglichen es unserem Entwicklungsteam, effizient und professionell an der Umsetzung der Lösung zu arbeiten.

Anlage 11 Installierte Symfony Bundles

Enabled Bundles (18)

Name	Class
ApiPlatformBundle	"ApiPlatform\Symfony\Bundle\ApiPlatformBundle"
DebugBundle	"Symfony\Bundle\DebugBundle\DebugBundle"
DoctrineBundle	"Doctrine\Bundle\DoctrineBundle\DoctrineBundle"
DoctrineFixturesBundle	"Doctrine\Bundle\FixturesBundle\DoctrineFixturesBundle"
DoctrineMigrationsBundle	"Doctrine\Bundle\MigrationsBundle\DoctrineMigrationsBundle"
FidryAliceDataFixturesBundle	"Fidry\AliceDataFixtures\Bridge\Symfony\FidryAliceDataFixturesBundle"
FrameworkBundle	"Symfony\Bundle\FrameworkBundle\FrameworkBundle"
HautelookAliceBundle	"Hautelook\AliceBundle\HautelookAliceBundle"
LexikJWTAuthenticationBundle	"Lexik\Bundle\JWTAuthenticationBundle\LexikJWTAuthenticationBundle"
MakerBundle	"Symfony\Bundle\MakerBundle\MakerBundle"
MonologBundle	"Symfony\Bundle\MonoLogBundle\MonoLogBundle"
NelmioAliceBundle	"Nelmio\Alice\Bridge\Symfony\NelmioAliceBundle"
NelmioApiDocBundle	"Nelmio\ApiDocBundle\NelmioApiDocBundle"
NelmioCorsBundle	"Nelmio\CorsBundle\NelmioCorsBundle"
SecurityBundle	"Symfony\Bundle\SecurityBundle\SecurityBundle"
TwigBundle	"Symfony\Bundle\TwigBundle\TwigBundle"
TwigExtraBundle	"Twig\Extra\TwigExtraBundle\TwigExtraBundle"
WebProfilerBundle	"Symfony\Bundle\WebProfilerBundle\WebProfilerBundle"

Abbildung 10 Installierte Symfony Bundles

Anlage 12 Fixtures Funktionen (Auszug)

```
src > DataFixtures > AppFixtures.php > ...
18 class AppFixtures extends Fixture
19 {
20     public function load(ObjectManager $manager): void
21     {
22         $this->saveFragetyp($manager);
23         $this->saveFrage($manager);
24         $this->saveUser($manager);
25         $this->saveUmfrage($manager);
26         $this->saveKommentar($manager);
27         $this->saveAntwort($manager);
28     }
29
30     1 reference | Codeium: Refactor | Explain | Generate Function Comment | X
31     private function saveFragetyp(ObjectManager $manager)
32     {
33         $fragetypOffen = (new Fragetyp())
34             ->setName("Offen");
35
36         $fragetypRadio = (new Fragetyp())
37             ->setName("Radio");
38
39         $manager->persist($fragetypRadio);
40         $manager->persist($fragetypOffen);
41         $manager->flush();
42     }
43
44     1 reference | Codeium: Refactor | Explain | Generate Function Comment | X
45     private function saveFrage(ObjectManager $manager)
46     {
47         $fragetyp = $manager->getRepository(Fragetyp::class)->find(1);
48         $frage = (new Frage())
49             ->setFrage("Kann wer am Dienstag Kuchen backen")
50             ->setFragetyp($fragetyp)
51             ->setKonfiguration(["antwortmoeglichkeiten" => ["a" => "ok", "b" => "nein"]]);
52
53         $manager->persist($frage);
54         $manager->flush();
55     }
56 }
```

Listing 2 Fixtures Funktionen (Auszug)

Anlage 13 Befehl zum Laden der Fixtures in die Datenbank

```
src > DataFixtures > AppFixtures.php > ...
1 <?php
2 // Fixtures neu Laden
3 /*
4 php bin/console --env=test doctrine:schema:drop --force ;
5 php bin/console --env=test doctrine:schema:update --force ;
6 php bin/console --env=test doctrine:fixtures:load -n
7 */
8
```

Listing 3 Befehl zum Laden der Fixtures in die Datenbank

Anlage 14 Testklasse User Registration

```
tests > UserTest.php > App\Tests\UserTest > testCreateUser()
0 references | 0 implementations | Codeium: Refactor | Explain
13 class UserTest extends ApiTestCase
14 {
    0 references | 0 overrides | Codeium: Refactor | Explain | Generate Function Comment | X
15 public function testLoginNotValid(): void
16 {
17     $client = static::createClient();
18     $client->request("POST", "/auth",
19         [
20             "json" => [
21                 "email" => "admin",
22                 "password" => "admin"
23             ]
24         ]
25     );
26     $this->assertResponseStatusCodeSame(401);
27 }
28
    0 references | 0 overrides | Codeium: Refactor | Explain | Generate Function Comment | X
29 public function testLoginValid(): void
30 {
31     $client = static::createClient();
32     $response = $client->request("POST", "/auth",
33         [
34             "json" => [
35                 "email" => "abc@a.de",
36                 "password" => "1"
37             ]
38         ]
39     );
40     $this->assertResponseStatusCodeSame(200);
41     $this->assertArrayHasKey("token", $response->toArray());
42 }
43
```

Listing 4 Testklasse User Registration

Anlage 15 Konfiguration JWT

```
config > packages > ! security.yaml
1 security:
2   # https://symfony.com/doc/current/security.html#c-hashing-passwords
3   password_hashers:
4     App\Entity\User: 'auto'
5   # https://symfony.com/doc/current/security.html#where-do-users-come-from-user-providers
6   providers:
7     # used to reload user from session & other features (e.g. switch_user)
8     users:
9       entity:
10         class: App\Entity\User
11         property: email
12
13   firewalls:
14     dev:
15       pattern: ^/_(profiler|wdt)
16       security: false
17     api:
18       pattern: ^/api/
19       stateless: true
20       provider: users
21       jwt: ~
22     main:
23       json_login:
24         check_path: /auth # nicht wie in der Dokumentation beschrieben (https://api-platform.com/docs/core/jwt/)
25         username_path: email
26         password_path: password
27         success_handler: lexik_jwt_authentication.handler.authentication_success
28         failure_handler: lexik_jwt_authentication.handler.authentication_failure
```

Listing 5 Konfiguration JWT

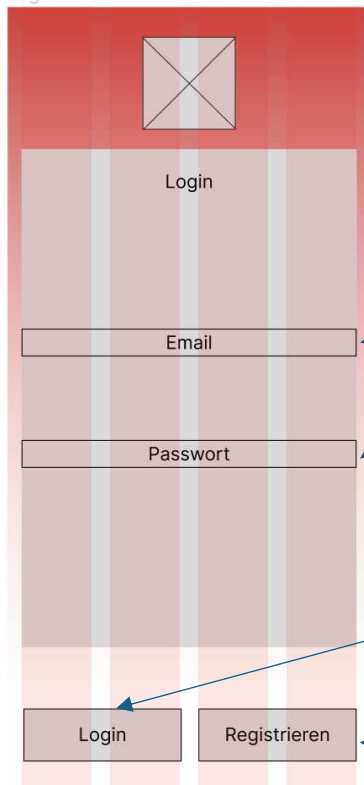
Anlage 16 Controller zur Registrierung

```
src > Controller > UserController.php > App\Controller\UserController
17
18 18 references | 0 implementations | Codeium: Refactor | Explain
19 class UserController extends AbstractController
20 {
21
22 Codeium: Refactor | Explain | Generate Function Comment | X
23 #[Route('/api/registerUser', name: 'app_user', methods: "POST")]
24 16 references | 0 overrides
25 public function registerUser(
26     Request $request,
27     EntityManagerInterface $manager,
28     UserPasswordHasherInterface $userPasswordHasher): JsonResponse
29 {
30     $requestJson = json_decode($request->getContent());
31
32     $email = $requestJson->email;
33     $password = $requestJson->password;
34     $name = $requestJson->name;
35
36     if (empty($email)) {
37         return $this->json(["message" => "missing email"], Response::HTTP_BAD_REQUEST);
38     }
39
40     if (empty($password)) {
41         return $this->json(["message" => "missing password"], Response::HTTP_BAD_REQUEST);
42     }
43
44     if (empty($name)) {
45         return $this->json(["message" => "missing name"], Response::HTTP_BAD_REQUEST);
46     }
47
48     if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
49         return $this->json(["message" => "email is not valid"], Response::HTTP_BAD_REQUEST);
50     }
51
52     if ($manager->getRepository(User::class)->findOneBy(["email" => $email])) {
53         return $this->json(["message" => "email already exists"], Response::HTTP_CONFLICT);
54     }
55
56     try {
57         $user = (new User())
58             ->setEmail($email)
59             ->setName($name)
60             ->setRoles(["ROLE_USER"])
61             ->setRegistrierDatum(new \DateTime());
62
63         $user->setPassword($userPasswordHasher->hashPassword($user, $password));
64         $manager->persist($user);
65         $manager->flush();
66         return $this->json([$user], Response::HTTP_CREATED);
67     } catch (\Exception $e) {
68         return $this->json([$e->getMessage()], Response::HTTP_BAD_REQUEST);
69     }
70 }
```

Listing 6 Controller zur Registrierung

Anlage 17 Anwenderdokumentation

Login



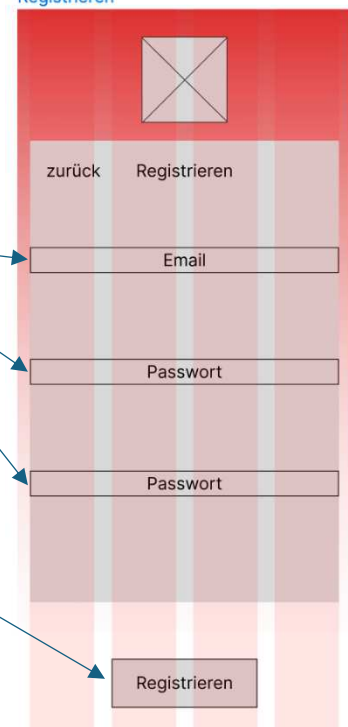
The login screen features a red header with a placeholder for a logo. Below the header is a white box with the title 'Login'. Underneath the title are two input fields labeled 'Email' and 'Passwort'. At the bottom of the screen, there are two buttons: 'Login' and 'Registrieren'.

Bitte tragen Sie hier
die erforderlichen
Daten zum Anmelden

Nach Eingabe der Login-Da-
ten können Sie sich hier
einloggen und gelangen
zum Home-Screen.

Bitte nutzen Sie dieses
Feld zur Weiterleitung
zum Registrierungsfor-
mular.

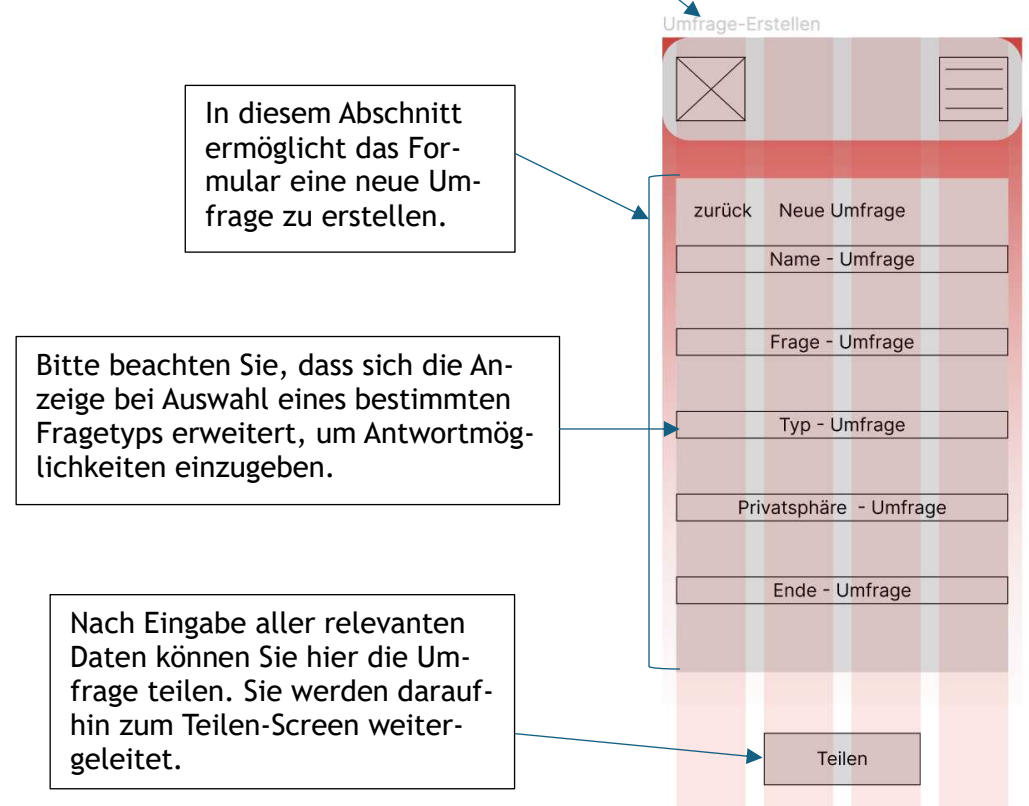
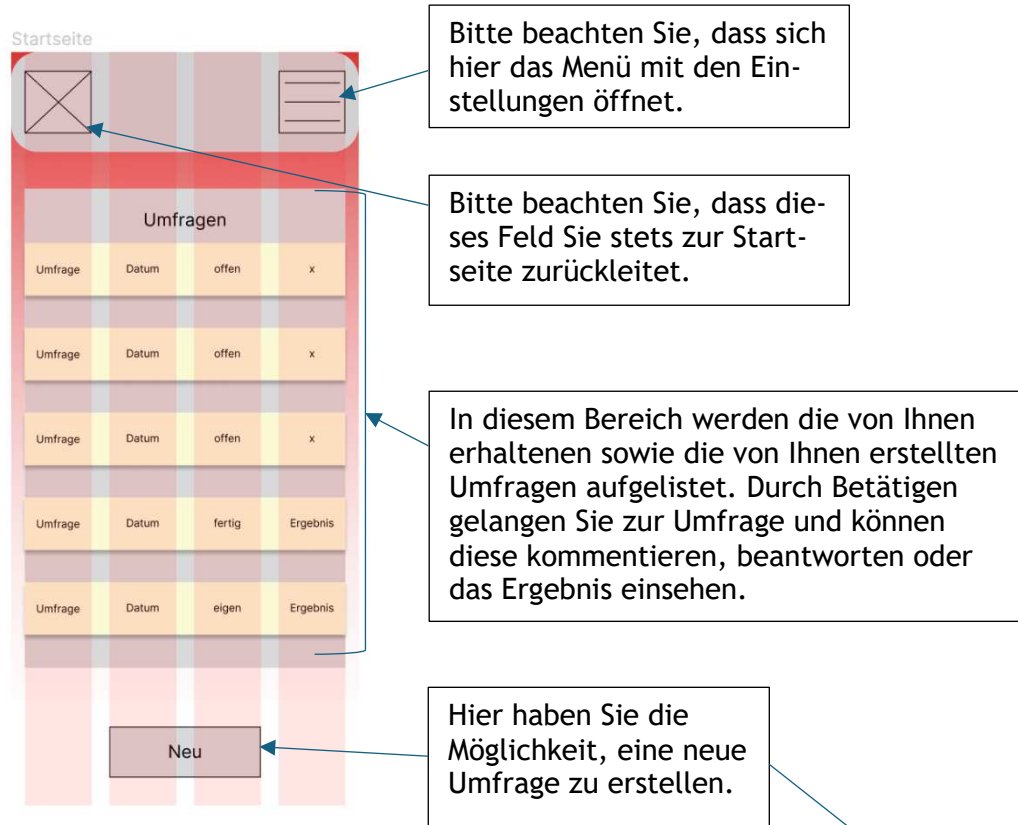
Registrieren



The registration screen has a red header with a placeholder for a logo. Below the header is a white box with the title 'Registrieren'. Underneath the title are three input fields labeled 'Email', 'Passwort', and 'Passwort'. At the bottom of the screen, there is a button labeled 'Registrieren'. A 'zurück' button is located at the top left of the white box.

Bitte beachten Sie, dass
zur Registrierung die er-
forderlichen Daten bereit-
gestellt werden müssen.

Durch Betätigen der Schalt-
fläche wird der Nutzer regis-
triert und anschließend zum
Login weitergeleitet. Im An-
schluss besteht die Möglich-
keit, sich mit den hinterleg-
ten Daten anzumelden.



Umfrage-Teilen

zurück Teilen

Text

Link

Kopieren

Hier finden Sie Ihre gestellte Frage und den Link, den Sie gern teilen dürfen.

Mit diesem Feld wird der Link kopiert.

Hier können Sie eine Erhaltene Umfrage Beantworten. Nachdem die Umfrage beantwortet wurde, ist es möglich die Auswertung einzusehen.

Sind Sie Der Ersteller ist es möglich hier diese Umfrage zu Löschen oder direkt zur Auswertung zu gelangen.

Umfrage-Antwort

zurück Antworten

Status

Antwortmöglichkeit A in %

Antwortmöglichkeit B in %

Im Unteren Bereich ist es Ihnen Möglich die Umfrage zu Kommentieren.

Umfrage-Beantworten

zurück Frage beantworten

Text

Antwortmöglichkeit A

Antwortmöglichkeit B

Beantworten

Kommentare

Text

Text

Text

Kommentieren ...

Anlage 18 API-Dokumentation

GET

/api/umfragen

Retrieves the collection of Umfrage resources.

Retrieves the collection of Umfrage resources.

Parameters

Try it out

Name	Description
page integer (query)	The collection page number Default value : 1 1
titel string (query)	titel
ersteller.id integer (query)	ersteller.id
ersteller.id[] array(integer) (query)	
frage.fragetyp.name string (query)	frage.fragetyp.name
frage.fragetyp.name[] array(string) (query)	
order[guelteigBis] string (query)	Available values : asc, desc -- ▾
order[guelteigAb] string (query)	Available values : asc, desc -- ▾

Responses

Code	Description	Links
200	Umfrage collection Media type application/ld+json ▾ Controls Accept header. Example Value Schema	No links

```
{
  "hydra:member": [
    {
      "id": "string",
      "etype": "string",
      "@context": "string",
      "id": 0,
      "beschreibung": "string",
      "link": "string",
      "titel": "string",
      "guelteigBis": "2024-05-23T19:51:50.838Z",
      "guelteigAb": "2024-05-23T19:51:50.841Z",
      "erstellt": "2024-05-23T19:51:50.841Z",
      "frage": "https://example.com/",
      "user": [
        "https://example.com/"
      ],
      "antworten": [
        "https://example.com/"
      ],
      "kommentare": [
        "https://example.com/"
      ],
      "ersteller": "https://example.com/"
    }
  ]
}
```

Abbildung 11 Auszug 1 API-Dokumentation

Umfrage			^
GET	/api/umfragen	Retrieves the collection of Umfrage resources.	🔒 ↓
POST	/api/umfragen	Creates a Umfrage resource.	🔒 ↓
GET	/api/umfragen/{id}	Retrieves a Umfrage resource.	🔒 ↓
PUT	/api/umfragen/{id}	Replaces the Umfrage resource.	🔒 ↓
DELETE	/api/umfragen/{id}	Removes the Umfrage resource.	🔒 ↓
PATCH	/api/umfragen/{id}	Updates the Umfrage resource.	🔒 ↓
GET	/api/users/{id}/erstellteUmfragen	Retrieves the collection of Umfrage resources.	🔒 ↓
GET	/api/users/{id}/umfragen	Retrieves the collection of Umfrage resources.	🔒 ↓
User			^
GET	/api/users	Retrieves the collection of User resources.	🔒 ↓
POST	/api/users	Creates a User resource.	🔒 ↓
GET	/api/users/{id}	Retrieves a User resource.	🔒 ↓
PUT	/api/users/{id}	Replaces the User resource.	🔒 ↓
DELETE	/api/users/{id}	Removes the User resource.	🔒 ↓
PATCH	/api/users/{id}	Updates the User resource.	🔒 ↓
Login Check			^
POST	/auth	Creates a user token.	🔒 ↓

Abbildung 12 Auszug 2 API-Dokumentation

Anlage 19 Soll-Ist-Vergleich

Projektphase	Soll Zeit	Ist Zeit	Differenz
Analyse	12 h	16 h	+ 4 h
Selbständige Fortbildung	7 h	9 h	+ 2 h
Absprache mit dem Projektleiter	2 h	2 h	± 0 h
Ist-Analyse	1 h	1 h	± 0 h
Wirtschaftlichkeitsanalyse	2 h	4 h	+ 2 h
Entwurf	15 h	15 h	± 0 h
Erstellung eines ERM und Datenbankmodels	3 h	4 h	+ 1 h
Erstellung des Pflichtenhefts	4 h	5 h	+ 1 h
Erstellung eines Geschäftsprozessdiagramm	4 h	4 h	± 0 h
Festlegen der API-Endpunkte	4 h	2 h	- 2 h
Implementierung inkl. Tests	41 h	35 h	- 6 h
Erstellung der Datenbank	3 h	2 h	- 1 h
Implementierung der Backend-Funktionen	30 h	24 h	- 6 h
Testphase	5 h	3 h	- 2 h
Debugging	3 h	6 h	+ 3h
Abnahme	2 h	2 h	± 0 h
Qualitätssicherung	2 h	2 h	± 0 h
Dokumentation	10 h	12 h	+ 2 h
Anwenderdokumentation	2 h	2 h	± 0 h
Technische Dokumentation	8 h	10 h	+ 2 h
Gesamt	80 h	80 h	± 0 h

Tabelle 4 Soll-Ist-Vergleich